Experimental Methodology

Lesson 5 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

6 November 2026 · reading time about 95 minutes

Contents

Lesson plan . . . 2
1. Why this lesson sits in the middle of the course . . . 3
    1.1 A promise we broke on purpose . . . 3
2. A test score is a measurement . . . 4
    2.1 The picture first . . . 4
    2.2 How wide the error bar actually is . . . 4
    2.3 Why the flattering results are the dangerous ones . . . 5
    2.4 The arithmetic behind the spread . . . 6
3. Selecting a model on one split selects noise . . . 7
4. Cross-validation . . . 10
    4.1 The idea . . . 10
    4.2 What it estimates, precisely . . . 11
    4.3 Why the usual error bar is optimistic . . . 13
    4.4 Stratification . . . 15
    4.5 How many folds . . . 16
    4.6 When the rows are not independent . . . 17
    4.7 How much it helps . . . 19
5. What the error is made of . . . 21
    5.1 The picture first . . . 21
    5.2 The bias–variance decomposition, derived . . . 22
    5.3 What it looks like on real numbers . . . 25
    5.4 The optimum depends on how much data you have . . . 28
    5.5 Without the truth: training error against validation error . . . 28
6. Learning curves . . . 30
    6.1 Reading them . . . 31
    6.2 The one-line diagnostic . . . 32
7. Leakage that survives cross-validation . . . 32
    7.1 The catastrophe . . . 32
    7.2 The fix, and the surprise . . . 33
    7.3 Hyperparameter search is the same problem . . . 34
    7.4 Nested cross-validation . . . 36
8. Doing lesson 4's threshold honestly . . . 40
9. Reproducibility . . . 41

9.1 Seeds . . . 41
9.2 The rest of it . . . 41
10. Summary . . . 41
What belongs in a report . . . 42
Homework . . . 42
Notation used in this lesson . . . 42

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 4 discussed: the threshold each student chose, and on which data it was chosen	Exercise 4, no slide
0:10–0:15	5	The promise we broke; the plan for today	Slides 2–3
0:15–0:35	20	One split is a lottery; parameters and hyperparameters; the validation set and its two recipes; choosing on one split (atom 1 with slide 9 if there are three minutes)	Slides 4–11
0:35–1:05	30	Cross-validation, what it estimates, the spread, stratification, grouped rows (18–19 optional); atoms 2 and 3 in place of notebook 01	Slides 12–23
1:05–1:15	10	**Break**	Slide 24
1:15–1:55	40	The question, the lab, the decomposition, the trade-off; without the truth; learning curves; which fix to buy (31–32, 34–35 optional); notebook 02 left for self-study	Slides 25–41
1:55–2:30	35	**Leakage that survives cross-validation**, the heart of the lesson	Slides 42–53

Time	Minutes	Segment	Material
2:30–2:40	10	The debt paid; three sets, three jobs (56–57 optional)	Slides 54–57
2:40–2:55	15	Atoms 11 to 13 in place of notebook 03	Slide 58
2:55–3:00	5	What to put in a report; what to take away; homework, exercise 5 set	Slides 59–61
	180	**Total**	**61 slides, 3 notebooks, the atoms**

The notebooks give way to short atoms from `Consolidation/05_lesson_5_atoms.ipynb`*, which students followed better than live notebooks in lesson 4: each atom asks for a prediction, then runs. Atoms 2 and 3 replace notebook 01 (slide 23), atoms 11 to 13 replace notebook 03 (slide 58), and notebook 02 (slide 41) is left for self-study. All three notebooks, the other atoms and the slides marked (optional) — 18 and 19, 31 and 32, 34 and 35, 56 and 57 — are for self-study; the sections of this handout that cover them are complete.*

---

1. Why this lesson sits in the middle of the course

You can already build models. Lessons 3 and 4 gave you regression and classification, and scikit-learn gives you a hundred more for one line each. **That is precisely the problem this lesson exists to solve.**

A student who can program will produce a model that appears to work within an afternoon. Producing a model that appears to work is not difficult and it is not the job. The job is knowing whether it *does* work, and that turns out to be considerably harder than fitting it — hard enough that published results in several fields have not survived scrutiny.

Everything after today depends on this. Lessons 6 to 10 add more powerful and more flexible methods, and every increase in flexibility increases the number of ways a result can be wrong while looking right.

1.1 A promise we broke on purpose

Lesson 4 ended by choosing a decision threshold — and choosing it by sweeping values on the **test set**, which lesson 1 forbade. We said at the time that this was a demonstration of a method rather than a reportable result, and that lesson 5 would fix it.

Section 8 fixes it. Keep the debt in mind while reading, because almost every error in this lesson has the same shape: **a choice was made by looking at data that was then used to report the result.**

---

2. A test score is a measurement

2.1 The picture first

You would not report the result of a single opinion poll of forty people to three decimal places. You would say: this is an estimate, it has an error bar, and the error bar is wide because forty people is not many.

A test score is exactly the same kind of object. It is computed on a finite sample of held-out rows, and a different sample would have given a different number. The rule from lesson 1 — hold data out, never look at it — makes the estimate *unbiased*. It does nothing whatever to make it *precise*.

Unbiased *for what*, precisely. The test score is an empirical risk $\hat{R}$, computed on the test rows. What it estimates without bias is the **expected risk** R of lesson 1: the error over the whole distribution $\mathcal{D}$, on all the drives the model will ever meet. Averaged over all the test sets one might have drawn, $\mathbb{E}[\hat{R}_{\text{test}}] = R(f)$ — and that holds because the test rows played no part in choosing f. *Precise* is a different property: how close a single measurement lands to R.

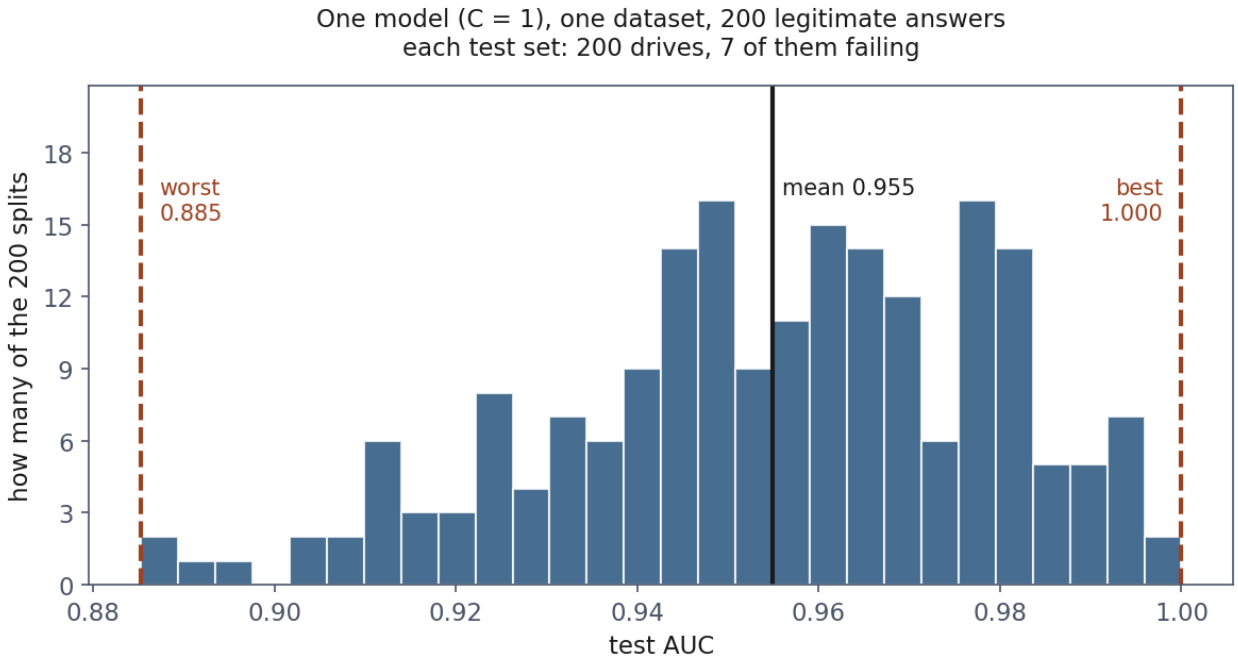
The poll in numbers: forty people, a proportion near one half, so a standard error of $\sqrt{0.25/40} = 0.079$. The second decimal is already uncertain; the third is decoration.

A predictable mistake is to call the test score "an unbiased estimator of the empirical risk". It is natural — both are averages over rows — but the test score *is* an empirical risk; what it estimates is the expected one.

That distinction is the whole of section 2, and it is routinely missed, because the phrase "test set accuracy" sounds like a property of the model rather than a measurement with a standard error.

2.2 How wide the error bar actually is

Take the disk fleet from lesson 4 and cut it to **800 drives, 29 of which fail** ($29 \div 800 = 3.6\%$, against lesson 4's 306 of 8,000, 3.8%). This is no exaggeration of the effect: a few hundred records with a few dozen positives is a far more ordinary situation than 8,000, and the numbers below are what that situation looks like when every step is done correctly. Real projects often start with a few hundred labelled rows and a few dozen of the rare class — a pilot study, a year of failures in a small fleet. At 8,000 drives a 25% test set held $306 \times 0.25 = 76.5$, so 76 failures in lesson 4's split; at 800 it holds 7.



Two hundred legitimate splits of the same 800 drives, the same logistic regression (C = 1) refitted on each, and each test set holding 200 drives of which 7 fail. Each bar is a bin about 0.004 of area under the curve (AUC) wide, (1.000 − 0.885) ÷ 28 bins, and its height is how many splits landed in it. Every one of these is a correctly performed experiment; the dashed lines are the worst and the best. The right-hand tail is the dangerous end — a flattering score is the one nobody goes back to question.

Split it 75/25, fit a logistic regression, and score it with the area under the receiver operating characteristic (ROC) curve (AUC) of lesson 4. Then do it again with a different random seed. Two hundred times. On each split the model is refitted on that split's training part, so these are 200 slightly different models from one procedure, and their mean is the procedure's:

	Test AUC
worst split	0.885
best split	**1.000**
mean	0.955
standard deviation	0.024

The best split reports a perfect classifier. All 7 failing drives ranked above all 193 healthy ones, on a model whose mean over the 200 splits is 0.955. Nothing was done wrong to obtain it: the split is legitimate, the model never saw the test rows, the metric is computed correctly.

Ten of the two hundred splits scored above 0.99.

2.3 Why the flattering results are the dangerous ones

A disappointing score makes you frown and keep working. **A delightful score makes you stop.** You write it down, put it in the report, and move on to the next thing — and the splits that flatter you are precisely the ones you are least likely to interrogate.

This is not a claim about dishonesty. It is a claim about attention: the errors that survive are the ones nobody had a reason to look for.

2.4 The arithmetic behind the spread

Each test set here contains **seven failures**. Every AUC quoted above is computed from seven positive examples against 193 negatives.

Why seven is the number that matters, rather than two hundred. Take recall, the fraction of failures the model catches. It is a proportion computed over the failures and over nobody else: each of the m_+ positive test rows is either caught or missed, so the count caught is a sum of m_+ independent yes/no draws, and the proportion estimated from it carries the standard error lesson 1, Section 2.4 derived for exactly this situation:

$$\mathrm{SE} = \sqrt{\frac{p(1-p)}{m_+}}$$

Look at what is not in that expression. The healthy rows appear nowhere. Adding ten thousand of them leaves the error bar on recall precisely where it was, because recall was never measured on them. It is m_+, and only m_+, that buys precision — which is the arithmetic under the rule of thumb worth carrying: **when one class is rare, the precision of a metric computed on it — recall here, and in practice AUC, every pair of which needs a positive — is governed by the count of the rarer class, not by the size of the dataset.** A metric computed on the negatives, such as specificity, depends on their count instead, and with 771 healthy drives that is the one that comes cheap.

Put the three fleet sizes through it, at a recall of about 0.9:

Positives	SE of recall
7, this test set	**0.113**
29, the whole cut-down fleet	0.056
306, the full fleet	0.017

Eight thousand drives with 3.8% failures gives a stable estimate; eight hundred does not, and the difference is 306 positives against 29 — a factor of ten in the count, and a factor of $\sqrt{10}$, about three, in the precision.

AUC is not a proportion of that kind, but it is governed by the same count for the same reason. It is an average over the $m_+ \times m_-$ positive–negative pairs, and those pairs are nowhere near independent: one positive example appears in m_- of them, so a single unlucky positive moves many pairs at once. Seven positives are seven pieces of evidence however many healthy drives accompany them.

Worked through. Here $m_+ = 7$ and $m_- = 193$, so there are $7 \times 193 = 1{,}351$ pairs, in seven bundles of 193, one bundle per failing drive. Suppose one failing drive is scored a little lower on another split and slips below 20 more healthy drives than before. Twenty of its pairs turn from right to wrong, and the AUC falls by

$$\frac{20}{m_+ m_-} = \frac{20}{1{,}351} = 0.015,$$

over half the standard deviation of 0.024 measured across the 200 splits. One drive, ranked slightly differently, is enough. The rule is AUC = pairs ranked right ÷ 1,351, so in numbers: a perfect ranking is 1,351 ÷ 1,351 = 1.000, and twenty pairs turned wrong give 1,331 ÷ 1,351 = 0.985. The twenty is an illustration — about a tenth of the 193 healthy drives — not a measured value.

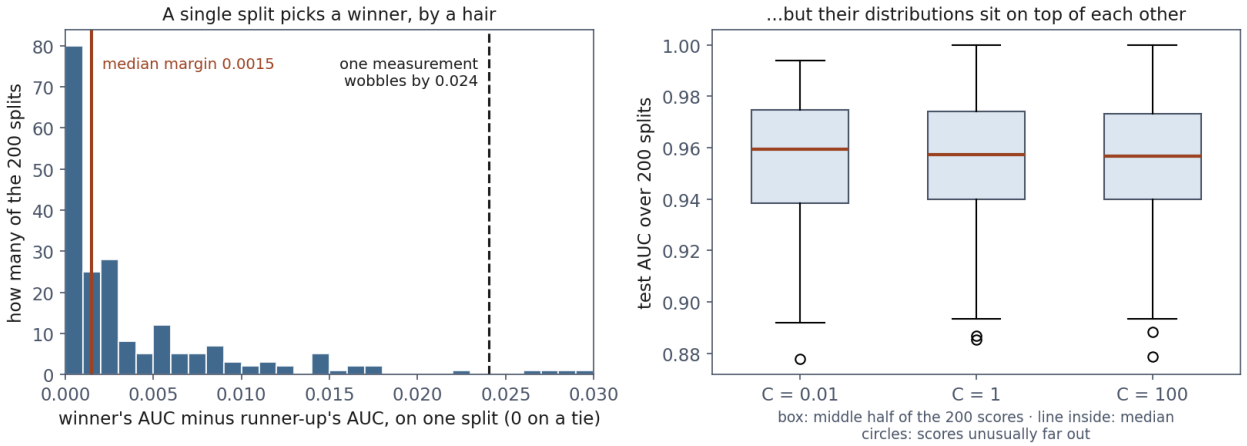
Two predictable mistakes. "Each test set holds 7% failures": it holds seven failures, 7 ÷ 200 = 3.5%, the fleet's 3.6% within rounding. And "the AUC counts how many times a failing drive outscores a healthy one": it is that count divided by 1,351, a fraction between 0 and 1.

If you remember one diagnostic question from this lesson, make it: *how many positive examples are in the test set?*

---

3. Selecting a model on one split selects noise

Reporting a noisy number is bad. **Choosing** with a noisy number is worse, because noise does not average out when it is used to make a decision — it decides.



Left: what each winner wins by — on each of the 200 splits, the winner's AUC minus the runner-up's, 0 on a tie. The median margin is 0.0015 (red); the dashed line is the 0.024 a single measurement wobbles by, sixteen times wider. Right: the 200 test AUCs of each model as a box plot — the box holds the middle half of the scores, the line inside it is the median, the whiskers reach the rest and the circles mark scores unusually far out. The three boxes sit at the same height. The right panel is the truth; the left is how little separates the winner a single experiment reports.

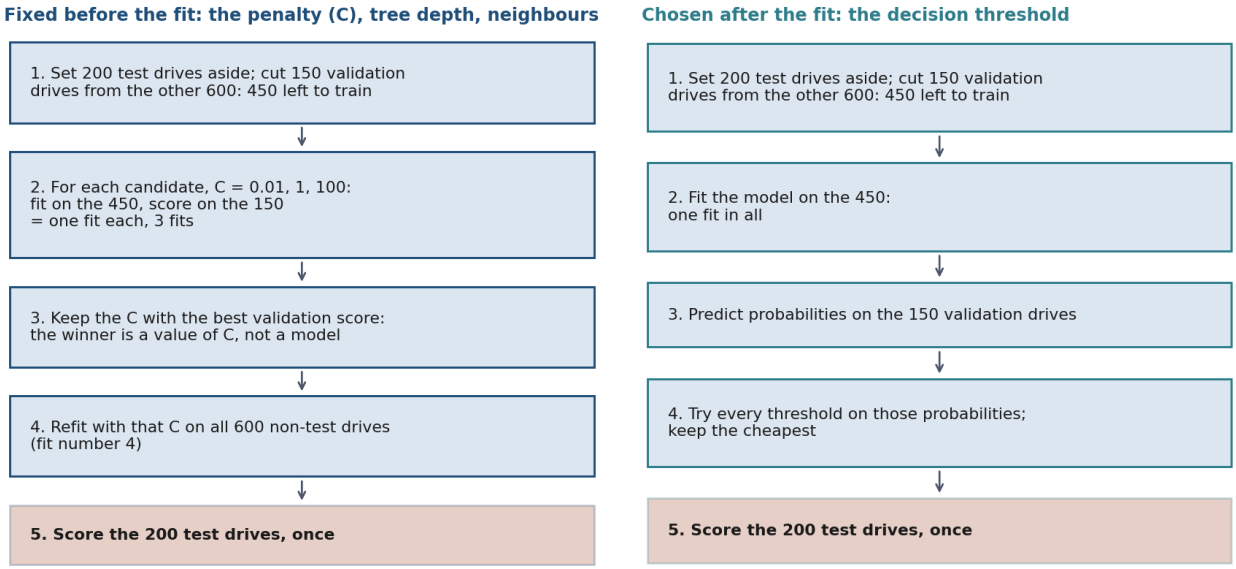
First the word, because the rest of the lesson leans on it. A fitted model holds two kinds of number. The ones the fit **estimates** from the data — the coefficients w and the intercept b of lessons 3 and 4 — are its **parameters**. The ones it does not estimate are its **hyperparameters**: they govern *how* the fitting is done, or how its output becomes a decision, and they are chosen from outside it. The penalty strength λ of lesson 3, the number of neighbours in lesson 6 and the depth of a tree in lesson 7 govern the fitting; lesson 4's decision threshold governs the decision.

Not "fixed before the fit", which is the tempting shorthand. λ does have to be fixed first, because the fit minimises a cost that contains it. The threshold is the counter-example: it is chosen *afterwards*, on the probabilities the fit has already produced. What the two have in common is not their timing but that neither is estimated the way a coefficient is.

And neither could be. Estimating a hyperparameter by minimising the training error would choose the value that fits the training data best — $\lambda = 0$ every time, the largest tree available, and a threshold tuned to the rows already seen. So they are chosen by **comparison** instead: fit or score the model once per candidate value and see which one generalises. That comparison needs data the model did not train on, which is why the rest of this lesson is about how to spend it.

The validation set: a test set for choosing. Training error cannot make the comparison — it always prefers the most flexible model — and the test set must not, or it is spent: a number chosen on is no longer a measurement (lesson 4's threshold, section 8). So a part of the data that is not the test set is held out for choosing, the **validation set**. Three sets, three jobs: the training set fits the parameters, the validation set makes every choice, the test set reports once. The sizes below are section 8's split of the 800 drives: 200 to test, 150 to validate, 450 to train.

Two recipes, by where the hyperparameter acts.



Both recipes on a fixed validation set. Left, a hyperparameter inside the cost: every candidate needs its own fit on the 450, is scored on the 150, and the winning value — a number, not a model — is refitted once on all 600 non-test drives before the single test score. Right, a hyperparameter acting on the output: one fit on the 450 serves every candidate. Both touch the 200 test drives once, at the end.

A hyperparameter fixed *before* the fit — λ, a tree's depth, the number of neighbours:

1. Set the 200 test drives aside, and cut 150 validation drives from the other 600, leaving 450 to train.
2. For each candidate, fit on the 450 and score on the 150: three values of `C` cost three fits.
3. Keep the candidate with the best validation score. What wins is a **value** of the hyperparameter, not a model: the three fitted models are discarded.

4. Refit with it on all 600 non-test drives, training plus validation: fit number 4.
5. Score the test set, once.

Section 4 replaces the single validation set with cross-validation, and the recipe stays the same with one change in step 2: with 5 folds each candidate costs five fits, so three values of `C` cost $3 \times 5 = 15$ fits, and the refit of step 4 is fit number 16. scikit-learn's `GridSearchCV` (section 7.3) runs exactly that loop: with `refit=True`, its default, `best_estimator_` is the refitted model of step 4.

A hyperparameter chosen *after* the fit — the decision threshold:

1. Set the test set aside, and cut a validation set from the rest.
2. Fit the model once, on the training part.
3. Predict probabilities on the validation set.
4. Try every threshold on those probabilities and keep the cheapest.
5. Score the test set, once. Section 8 does it with lesson 4's costs.

Two predictable mistakes. "Hyperparameters are chosen before the fit": the threshold is chosen after it; what they share is that the fit does not estimate them. And "fit once on the training data, then tune every hyperparameter on that model": true for the threshold, false for λ, which needs one fit per candidate. And a third, natural after step 3: "identify the winning model, then refit it". The winner is a value of the hyperparameter; the refit is a new model, fitted on more data than any candidate saw.

One warning about the notation, because scikit-learn inverts it. `LogisticRegression` is regularised through `C`, which is the *reciprocal* of the penalty: small `C` means a strong penalty, large `C` means almost none. It runs in the opposite direction to lesson 3's λ, and the table below is in `C`, so its first row is the most heavily penalised model, not the least. On this data the difference is visible in the coefficients: fitted on all 800 drives, `C = 0.01` gives a coefficient vector of length 0.52, and `C = 100` one of length 2.45.

Take three logistic regressions differing only in that one hyperparameter, and let a single split pick the winner. Two hundred times:

Model	Mean AUC	Standard deviation	Splits won outright
C = 0.01	0.9549	0.0240	89
C = 1	0.9549	0.0240	42
C = 100	0.9544	0.0243	38

One split, worked. On each split the three models are ranked by their test AUC; the margin is the winner's AUC minus the runner-up's, and the third model does not count. Seed 1, the first split without a tie:

Model	Test AUC on split seed 1	Rank
C = 0.01	0.9756	1, the winner
C = 100	0.9667	2, the runner-up
C = 1	0.9660	3
Margin = winner − runner-up	$0.9756 - 0.9667 = 0.0089$	

On this split C = 0.01 "wins" by 0.0089 — more than the median margin of 0.0015, and still under

half of the 0.024 by which a single measurement wobbles. The histogram on the left of the figure above is this subtraction, done on all 200 splits.

The outright wins add up to $89 + 42 + 38 = 169$; the other 31 splits are ties. AUC moves in steps of $1 \div 1{,}351$, and two similar models can rank the 200 test drives in exactly the same order: 29 of the ties are between C = 1 and C = 100, both barely penalised, one between C = 0.01 and C = 1, one three-way.

Read the first column, then the last.

The three models are equally good. Their mean performance agrees to three decimal places — a difference an order of magnitude smaller than the spread of a single measurement.

And yet a winner is declared on almost every split, by a hair. The winner's margin over the runner-up has a median of 0.0015 of AUC (mean 0.0039, largest 0.0296), against the 0.024 a single measurement wobbles by. The decision is made inside the noise.

The counts are not pure chance either: C = 0.01 wins most on other seeds too (seeds 200 to 399 give 86, 41 and 41 outright wins). It ranks the drives a little differently from the two near-twins, so it wins or loses alone while C = 1 and C = 100 share their wins. The counts say the models are *different*; they say nothing about which is *better* — for that, the means, and the means are equal.

Run this once, which is what real life allows, and you will report that one of these three is best, with a number to support it. **That conclusion is noise wearing the clothes of a result.**

---

4. Cross-validation

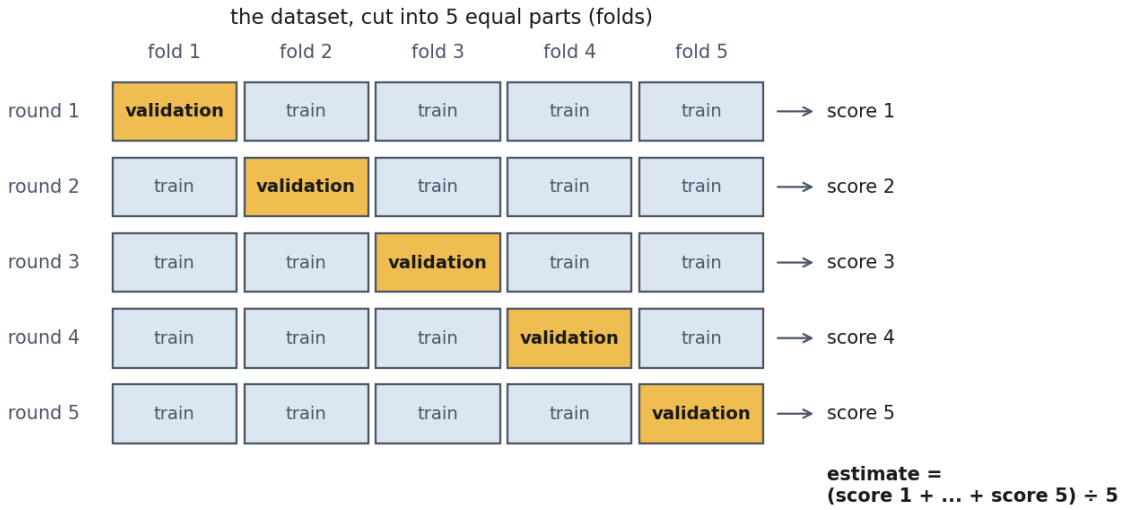
4.1 The idea

The picture first. If one measurement is noisy, take several and average them. The difficulty is that you cannot afford several test sets — you have one dataset, and every row held out is a row not spent on training.

Cross-validation resolves the tension by **rotating which part is held out**. Cut the data into k equal parts. Train on $k - 1$ and score the model on the one left out, the **validation fold**. Repeat until every part has been the validation fold exactly once. Here: 800 drives in 5 folds of 160, so each round trains on 640 drives and scores the 160 held out.

Two words, kept apart. A **fold** is a part of the data. A **round** is one fit and one score: one model per round. With k-fold there are as many rounds as folds, which is why "one model per fold" is said — **a predictable mistake** that later makes "every fold trains on 80%" sound as if a fold were a model. scikit-learn calls a round a *split*: `n_splits`.

The result is k scores, and — this is the part that makes it legitimate rather than a trick — **every row has been predicted exactly once, by a model that never saw it.** It is not testing on training data with extra steps.



Each column is a fold, each row a round. In each round the gold fold is held out, a model is fitted on the other four and scored on it; the five scores are averaged into the estimate. Every row is predicted exactly once, by a model trained without it — which is what makes this legitimate rather than testing on training data with extra steps.

Why *validation* and not *test*. Lesson 1 gave the three roles: training fits, validation chooses, test reports once. The block that rotates here is scored k times, and from Section 7.3 on it is what a search chooses on — so it is a validation fold. The code will call it something else: scikit-learn's `KFold.split` returns `train_index, test_index`, and `GridSearchCV` reports `mean_test_score`. There *test* means only *not used to fit this round's model*, never *the test set you report once*. When a final test set is needed it is set aside before cross-validation begins, and Section 7.4 shows where it goes.

Where it runs, and its two uses. Cross-validation runs on all the data that is not the test set: it *replaces* the fixed validation set. "It runs on the validation set" is **a predictable mistake**. To **measure** a procedure, with nothing being chosen, it runs on all the data and needs no test set at all: the model you ship is the same recipe refitted on all 800 drives, and the cross-validated mean is its estimate. To **choose** — a hyperparameter, a threshold — the test set is set aside first, cross-validation runs on the rest, the winner is refitted, and the test set is scored once (section 3's left recipe).

4.2 What it estimates, precisely

Let $A(S)$ denote the model that our training procedure produces from a dataset S, and let $R(f)$ be the expected error of a fixed model f on new data from the distribution $\mathcal{D}$. Cross-validation computes

$$\hat{R}_{CV} = \frac{1}{k}\sum_{j=1}^{k} \hat{R}_{S_j}\big(A(S \setminus S_j)\big)$$

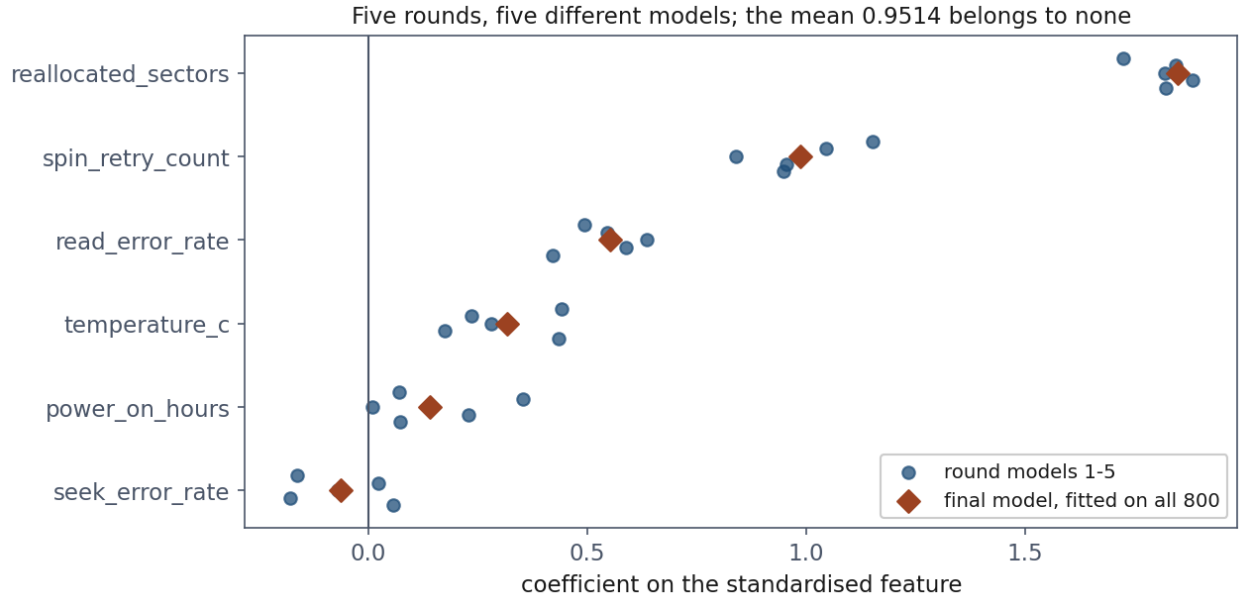
where S_j is the j-th fold. Each term is an unbiased estimate of the risk of a model trained on $m(1-1/k)$ examples.

Two consequences follow, and both are worth stating because both surprise people.

Cross-validation estimates the performance of the *procedure*, not of a particular fitted model. There are k different fitted models in the calculation and the reported number belongs to none of them. It answers: *if I apply this training procedure to a dataset of about this size, what should I expect?*

The five models are visible. At seed 0, 5-fold, C = 1, with coefficients on standardised features (each round fits its own scaler too, which moves them only slightly):

Model	AUC on its fold	reallocated sectors	spin retries	read errors	temperature	power-on hours	seek errors
round 1	0.929	1.723	1.152	0.494	0.442	0.070	−0.163
round 2	0.958	1.844	1.044	0.545	0.236	0.353	0.024
round 3	0.984	1.819	0.839	0.637	0.281	0.010	−0.069
round 4	0.939	1.881	0.955	0.588	0.175	0.228	−0.178
round 5	0.947	1.822	0.948	0.422	0.435	0.072	0.057
final, all 800	none	1.848	0.987	0.553	0.317	0.140	−0.063



One row per feature: the five blue dots are the five round models, the red diamond the model you would ship. Look at power-on hours, from 0.010 to 0.353, and at seek errors, which cross zero. The diamond sits among the dots, but no round ever scored it.

The models differ on individual drives too: at threshold 0.5 they flag 15, 17, 13, 14 and 13 drives and disagree on 8 of the 800. So 0.9514 is the mean of five scores of five different models, and the shipped model is a sixth that no round scored. The number is a property of the recipe — scale, then logistic regression with C = 1, trained on 640 drives. Section 4.7 adds a second piece of evidence: forty seeds give forty sets of five models, and the mean barely moves.

Which model do you ship, then? Keep two words apart. The **recipe** is the instructions —

standardise, then logistic regression with C = 1 — with no numbers in it. A **model** is the recipe fitted to some data: six coefficients and an intercept, the thing that calls `predict` on a new drive. Measuring and shipping are then two separate phases:

```
model = make_pipeline(StandardScaler(), LogisticRegression(C=1))    # the recipe

# 1. Measure the recipe: five temporary models, fitted, scored, discarded
scores = cross_val_score(model, X, y, scoring="roc_auc",
                         cv=StratifiedKFold(5, shuffle=True, random_state=0))
scores.mean()                        # 0.9514: what this recipe is worth

# 2. Build the model you ship: the same recipe, once, on all 800 drives
model.fit(X, y)
```

`cross_val_score` returns scores and no model. Five test cakes, each baked with four fifths of the ingredients, judge a recipe; the cake brought to the wedding is a sixth, baked with all of them. **A predictable mistake** is "the model you ship is one of the five, or their average": the red diamond sits among the dots, which invites it, but it is a new fit, and the five only measured the recipe. The five scores do not change that sixth fit at all: the recipe was fixed before, so it is refitted on all 800 whatever they said. If the scores change nothing, they are the measurement; if they change anything, they have made a choice.

And a second one, about deciding on the score. "If the cross-validated score is not acceptable, change the recipe and cross-validate again." Each round of that is a choice made on the same folds; keep the best of several recipes and its score is inflated, by the mechanism of section 7.3. Cross-validation without a test set is honest for one recipe decided in advance and measured once. If more than one will be tried, set the test set aside before the first.

The estimate is usually slightly pessimistic. Every round's model trains on a fraction $1-1/k$ of the data, and less training data usually means a slightly worse model — usually, because on a finite sample a learning curve need not rise at every step. So $\widehat{R}_{CV}$ estimates the risk of a model trained on 80% of your data (for $k = 5$), while the model you eventually ship is trained on 100% of it. The bias is in the safe direction, and it shrinks as k grows — which is the argument for leave-one-out cross-validation, where $k = m$.

4.3 Why the usual error bar is optimistic

The question. Cross-validation gave 0.9514 on this fleet. On another fleet of 800 drives, how far would that number move? That is the question a confidence interval answers.

First, what a confidence interval is. The standard error of an estimate is how far it would move if the whole experiment were repeated on a new dataset — lesson 1, section 2.4, and the poll of section 2.1. An approximate 95% confidence interval is the estimate plus or minus 1.96 standard errors: built that way, it contains the true value in about 95 of 100 repeated experiments. Approximate, because it leans on the normal curve and on a standard error that is itself right.

Having k scores, the temptation is to quote a standard error $s/\sqrt{k}$ — s the standard deviation of the k round scores — and treat it as a confidence interval. Here that gives $0.0187 \div \sqrt{5} = 0.0084$, so $0.9514 \pm 1.96 \times 0.0084 = 0.9514 \pm 0.016$. **Do not**, and the reasons are specific.

That formula assumes the k scores are independent, and nothing guarantees it: any two of the training sets share a fraction $(k-2)/(k-1)$ of their rows — for $k = 5$, three quarters of the

data is common to any pair — so the scores *can* be correlated. Positively correlated measurements carry less information than independent ones — and what that costs is worth deriving rather than asserting, because the derivation says exactly where the missing term went.

Write σ^2 for the variance of a single round score and ρ for the correlation between any two of them; the folds being interchangeable, one number serves for every pair. The variance of a sum collects every variance *and every covariance*:

$$\operatorname{Var}\left(\sum_{j=1}^{k}\hat{R}_{S_j}\right)=\sum_{j}\operatorname{Var}\left(\hat{R}_{S_j}\right)+\sum_{i\neq j}\operatorname{Cov}\left(\hat{R}_{S_i},\hat{R}_{S_j}\right)=k\sigma^2+k(k-1)\,\rho\sigma^2$$

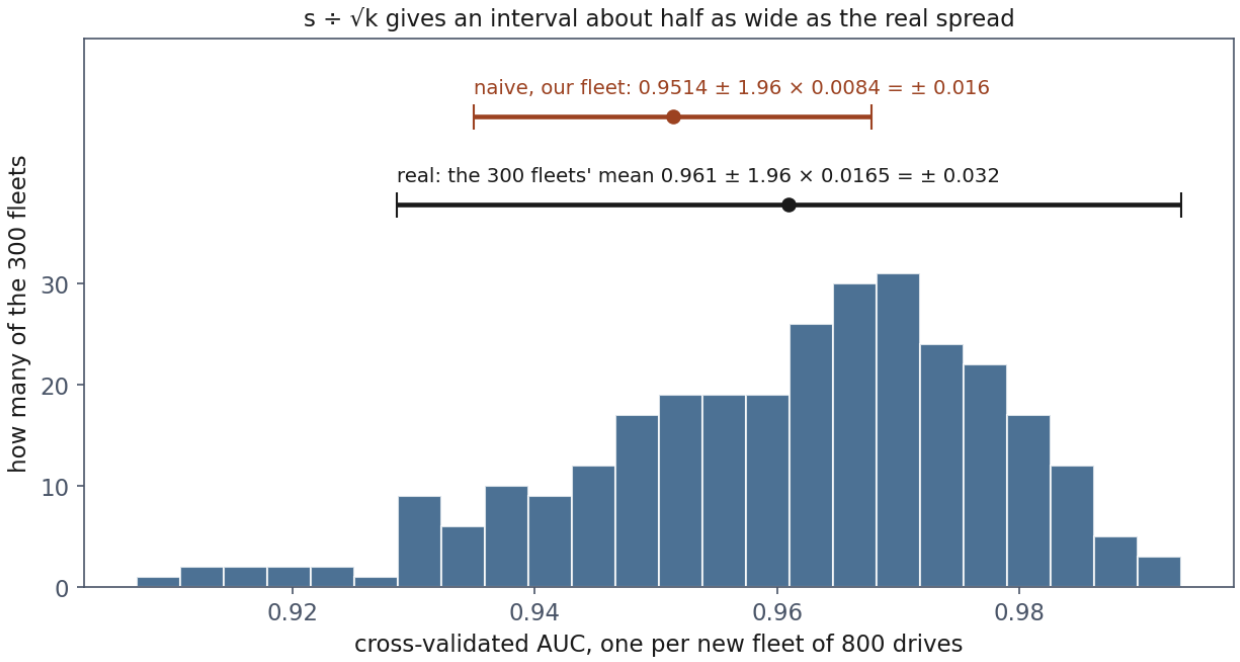
k variances, and $k(k-1)$ covariances because every ordered pair of distinct folds contributes one, each equal to $\rho\sigma^2$ by the definition of correlation. Cross-validation reports the *mean* of those k scores, and dividing a quantity by k divides its variance by k^2:

$$\operatorname{Var}\left(\hat{R}_{CV}\right)=\frac{k\sigma^2+k(k-1)\rho\sigma^2}{k^2}=\frac{\sigma^2}{k}+\frac{k-1}{k}\,\rho\,\sigma^2$$

Setting $\rho=0$ recovers the familiar σ^2/k, which is the assumption being made whenever it is used. The naive formula keeps only the first term. With ρ appreciably above zero the second term dominates, and no unbiased estimator of this variance exists in general — a result of Bengio and Grandvalet (2004).

And s itself is rough. It is computed from only $k=5$ numbers, so it is a poor estimate of σ, too small more often than not. That is the second reason the naive interval is too narrow, and on this fleet it is the larger one.

Measured. Notebook 1 does what real data never allows: it draws 300 new fleets of 800 drives from the same generator (seeds 1000 to 1299) and runs the same 5-fold cross-validation on each.



The histogram is the 300 cross-validated AUCs. The black bar is their real spread, ± 1.96 × 0.0165 = ± 0.032; the red bar is what $s \div \sqrt{k}$ promised on our fleet, ± 0.016 — half as wide. Compare the widths, not the centres: 0.9514 is our fleet's cross-validation, 0.961 the mean over the 300 new fleets.

	Value	How it is obtained
naive standard error, our fleet	0.0084	$0.0187 \div \sqrt{5}$
real standard deviation of the cross-validated mean	0.0165	over the 300 new fleets
correlation between two round scores, ρ	0.023	over the 300 new fleets
naive 95% interval contains the average of the 300	80% of fleets	$\pm 1.96\, s/\sqrt{5}$ on each fleet
the same with the t value for 4 degrees of freedom	88% of fleets	$\pm 2.776\, s/\sqrt{5}$

Two things to read off it. The interval is about half as wide as it should be: two models at 0.951 and 0.935 look separated at ±0.016 and are not at ±0.032. And on this data the correlation is small: each round score is dominated by its own 160 test drives, about six of them failing, and those are disjoint from the other rounds'. The overlap of the training sets makes the *models* similar; it need not make the *scores* correlated. Here the narrow interval comes mostly from s resting on five numbers; on data where the model part of the error matters more, ρ grows and the first reason takes over.

A predictable mistake: "the models train on correlated data, so the scores are correlated". The training sets overlap, but each score is measured on its own disjoint fold — here $\rho = 0.023$.

Another predictable mistake: comparing $s = 0.0187$ with 0.0165, nearly equal, and concluding that the formula works. They are spreads of different things: 0.0187 is how far one round's score moves from another round's on one fleet; 0.0165 is how far the *mean* of the five rounds moves from one fleet to the next. The fair comparison is between two spreads of the mean, 0.0084 against 0.0165, the first two rows of the table.

Checks. The $s = 0.0187$ above is the standard deviation of the five round scores divided by 5, NumPy's default (`std()`, `ddof=0`). Dividing by 4 instead (`std(ddof=1)`, the usual sample standard deviation) gives 0.0209, and $0.0209 \div \sqrt{5} = 0.0094$. The real spread, 0.0165, is then about 1.8 times the naive one rather than twice; the conclusion does not change. Neither version is a valid standard error for five dependent fold scores.

In practice: report the mean and the spread of the rounds, describe the spread as a spread rather than a confidence interval, and treat small differences between models as unresolved.

4.4 Stratification

With 29 positives among 800 rows, an unstratified 5-fold split can produce a fold containing almost none of them. Measured on the fleet:

Fold (seed 3)	Plain `KFold`	`StratifiedKFold`
1	7 failures of 160, AUC 0.979	5 of 160, AUC 0.983
2	**1 failure of 160, AUC 1.000**	6 of 160, AUC 0.969
3	7 of 160, AUC 0.942	6 of 160, AUC 0.902
4	5 of 160, AUC 0.965	6 of 160, AUC 0.945
5	9 of 160, AUC 0.943	6 of 160, AUC 0.969

The 29 failures are among all 800 drives, so a fold of 160 should hold $29 \div 5 = 5.8$ of them — as whole drives, 5 or 6. Both columns hold the same 29 in total ($7+1+7+5+9 = 5+6+6+6+6 = 29$); only the dealing differs. `StratifiedKFold` shuffles the failing and the healthy drives separately and deals each class into the folds in turn.

Fold 2 on the left holds **one** failing drive and scores a perfect 1.000: that drive happened to rank above all 159 healthy ones — 159 pairs, where a stratified fold has about $6 \times 154 = 924$. An AUC computed from a single positive example is defined, but it rests on one drive: all 159 pairs share it. Stratification does not promise a smaller spread on every seed — here the stratified scores spread a little more — it promises that no score rests on one drive. **A predictable mistake:** "each fold of 160 holds 29 failures". **Stratify whenever a class is rare** — which, per lesson 4, is whenever the problem is interesting.

4.5 How many folds

Larger k means each round's model trains on more data — 80% at $k = 5$, 90% at $k = 10$ — so the pessimistic bias of section 4.2 shrinks. **A predictable mistake** is "a larger k is more optimistic": cross-validation errs low, and a larger k errs less. It also means more fits, and training sets that overlap even more heavily — any two share 75% of their rows at $k = 5$ and 8/9, about 90%, at $k = 10$ — so the models are nearly the same and the round scores can be more strongly correlated (on this fleet the correlation is small, section 4.3). **A predictable mistake** is to blame the smaller validation fold for that correlation: it is the overlap of the training sets that makes the models alike.

Five or ten, in practice, and the difference between them rarely matters. $k = m$ is **leave-one-out**: every row is a validation fold on its own. It earns its place in one situation, **very few rows and a metric computed row by row** (mean squared error, accuracy): each model trains on $m-1$ rows, so the estimate is nearly unbiased, and there is only one way to cut, so no seed. With 30 patients, giving up a fifth of them to each validation fold costs a lot; giving up one costs almost nothing. The price is m fits, and an estimate that is notoriously high-variance: the m models are nearly identical and each score rests on a single row. With AUC it cannot work at all: a fold of one row holds no failing–healthy pair, so its AUC is undefined, and `cross_val_score` with `LeaveOneOut` returns `nan` for every round (for the same reason it cannot be stratified). On 800 drives scored by AUC, five or ten folds, or repeated folds. Nor does it work when rows come in groups, as in section 4.6: leaving out one of a drive's ten readings leaves the other nine, near-copies with the same label, in training, and the score is inflated. The extreme for grouped rows leaves out one whole group at a time, `LeaveOneGroupOut`.

k **is not a hyperparameter** — another predictable mistake. It is a setting of the *measurement*, not of the model: the model you ship is refitted on all the data whatever k was. Fix it beforehand, and never choose it by the score it gives.

If you have compute to spare, **repeated k-fold** — the whole procedure run with several different

shuffles and averaged; in scikit-learn `RepeatedStratifiedKFold(n_splits=5, n_repeats=10)` — buys more than moving from 5 to 10. Measured over 40 seeds on the fleet (C = 1, AUC):

Procedure	Fits	Mean of the 40 estimates	Standard deviation across seeds
5-fold	5	0.9532	0.0033
10-fold	10	0.9539	0.0036
5-fold, repeated twice	10	0.9530	0.0025
5-fold, repeated ten times	50	0.9533	0.0009

For the same ten fits, ten folds move the mean by $0.9539 - 0.9532 = 0.0007$ and do not reduce the spread; repeating five folds twice does. What repetition removes is the dependence on the fold cut. The uncertainty about new data, 0.0165 in section 4.3, stays. (The 0.0033 here is section 4.7's 0.0032 with $m - 1$ instead of m in the denominator.)

A predictable mistake: "ten repeats of 5-fold give 50 independent tests, so fifty times the evidence". The software returns 50 scores, and averaging more measurements usually buys precision. But all 50 reuse the same 800 drives: repetition removes the dependence on the fold cut, as the table shows, and nothing else.

4.6 When the rows are not independent

Everything above assumed the rows are independent draws from $\mathcal{D}$. That assumption is violated constantly, silently, and without any warning from the code.

The picture first. Suppose the table holds ten telemetry readings per drive rather than one. A random 5-fold split puts about eight of a drive's readings in the training set and two in the validation fold, in every round ($10 \times 4/5 = 8$ on average; the exact count varies from drive to drive). The model does not have to learn anything about failure: it learns to recognise *that drive*, and the test row is very nearly a duplicate of a training row.

The score is excellent. The model is worthless on a drive it has never seen — which is the only situation anyone cares about.

Why the test row is a near-duplicate: a drive's ten readings have nearly the same values and the same label, so a flexible model can bend its boundary around each drive it trains on, and a validation reading falls beside its siblings, on their side: it recognises the drive instead of learning the pattern. It is a form of leakage — the test row is not independent of the training rows — and it bites flexible models. Simulated on the fleet: each of the 800 drives given ten readings, its own values plus noise of 5% of each column's standard deviation (an assumption of the simulation), 8,000 rows, 5-fold AUC:

Model	Rows split at random	Each drive's readings kept together	One row per drive
logistic regression, C = 1	0.960	0.950	0.951

Model	Rows split at random	Each drive's readings kept together	One row per drive
logistic regression on degree-3 features, C = 10,000	**0.999**	**0.724**	0.775

The second model is the first made flexible with lesson 3's tools: degree-3 polynomial features turn the 6 columns into 84 (squares, cubes and products of the scaled columns), and C = 10,000 leaves almost no penalty, so the boundary is free to bend around each drive. Split at random it looks perfect; on unseen drives it is worth 0.72, far below the plain model. The plain logistic regression, with six coefficients, cannot memorise 800 drives and barely notices. Keeping each drive's readings together removes the lookup: each model lands near its one-row-per-drive score, not near its 0.999. The flexibility bought memory, not knowledge — lesson 3's overfitting, hidden by the split. **A predictable mistake:** "0.999 shows the flexible model is the better one"; split at random, it is graded on drives it has already seen.

The same structure appears whenever rows share a source: several images of the same patient, several sentences from the same document, repeated measurements of the same machine, any panel data.

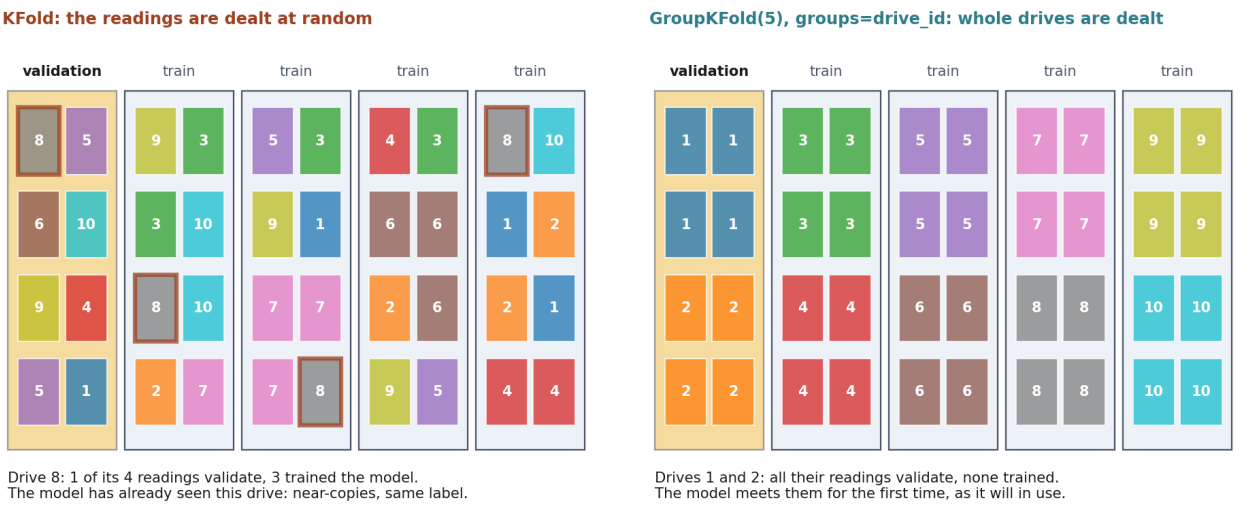
The rule: split along the axis you must generalise across.

How `GroupKFold` is called. It needs a group label per row, passed apart from the features. Here that label is `drive_id`: one value per row, the serial number of the drive the reading came from.

row	reallocated sectors	spin retries	…	failed (y)	`drive_id`
0	1.39	1.10	…	0	0
1	1.43	1.11	…	0	0
… rows 2 to 9, drive 0's other readings				0	0
10	1.60	−0.02	…	1	1
11	1.66	0.02	…	1	1
…					…
7,999	1.39	0.69	…	0	799

The first rows of the simulation, as atom 8 prints them: a drive's ten readings are nearly the same, with the same label.

`cross_val_score(model, X, y, groups=drive_id, cv=GroupKFold(n_splits=5), scoring="roc_auc")` puts all rows with the same `drive_id` in one fold: each round trains on 640 drives (6,400 readings) and validates on 160 others (1,600 readings), and no drive is on both sides.



Drawn small, ten drives with four readings each, one square per reading. Left, `KFold` deals readings at random: drive 8 has a reading in the validation fold and three in training, so the model is graded on near-copies of rows it learned. Right, `GroupKFold` deals whole drives: the drives that validate were never seen in training.

The label comes from the data — a serial number, a patient id — and it is *not* a feature: a model that sees it can recognise the drive again. Groups may differ in size: one drive with ten readings among drives with one each is split correctly. **A predictable mistake** is "GroupKFold needs equal-size groups". What unequal groups change is *weight*: that drive counts ten times in training and in the score, so aggregate to one row per drive or give its readings a weight of 1/10 (`fit(..., sample_weight=...)`), depending on whether the model will judge readings or drives.

Two cautions. `GroupKFold` is not stratified: with 29 failing drives a fold can get few of them, section 4.4's problem again; `StratifiedGroupKFold` does both. And the test set must be grouped too: `train_test_split` takes no groups, `GroupShuffleSplit` does.

One case this course does not treat: if the rows are ordered in time and the model must predict forward, validation must be forward too (`TimeSeriesSplit`), since no deployment trains on Wednesday to predict Tuesday.

The symptom of having needed `GroupKFold` and not used it is a validation score that is excellent and a production score that is not. It is among the most common reasons a model fails after deployment.

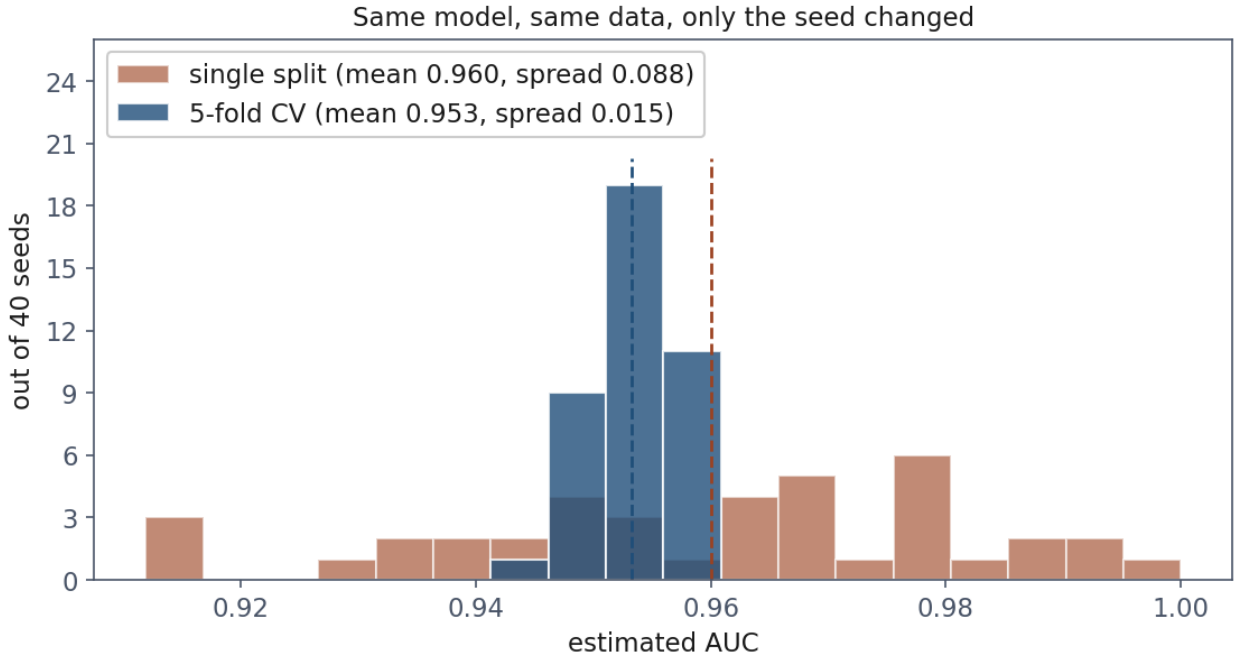
4.7 How much it helps

Cross-validation reduces the variance of the estimate. It does not abolish it: it has a random seed of its own, and shuffling differently cuts different folds and gives a different answer. So the honest comparison is **how far the number moves when nothing but the seed changes** — forty times, seeds 0 to 39, for a single split and for 5-fold cross-validation, the same C = 1 model throughout. The single-split column is the first forty of Section 2.2's two hundred splits, which is why its mean is 0.960 and not 0.955: forty draws centre less exactly than two hundred.

	Single 75/25 split	5-fold cross-validation
worst	0.9119	0.9439
best	1.0000	0.9588
mean	0.9600	0.9532
spread	0.0881	0.0149
standard deviation	0.0221	0.0032

Each number in the right-hand column is already a mean of five round AUCs, so the column is $40 \times 5 = 200$ fits.

Cross-validation is about seven times more stable across seeds. And both centre in about the same place: 0.960 against 0.953, a gap of about two standard errors on forty draws, which reverses sign with a different forty seeds. It is not more optimistic, it is less arbitrary.



Forty seeds, the same bins for both, so the bar heights compare directly. The dashed lines are the two means, close together; what differs is the width — every cross-validated estimate lands between 0.944 and 0.959, a range the single split spreads out from 0.912 to 1.000.

And the reason is not only the averaging. Averaging k measurements would buy a factor of at most $\sqrt{k}$, which is 2.2 here, and the observed factor is three times that. What the table really measures is how much the *seed* can move the answer, and the two procedures give it very different amounts of room. Changing the seed of a single split changes which quarter of the drives is tested — which seven of the 29 failures the score is computed from, and which rows the model was fitted on. It is a materially different experiment. Changing the seed of a cross-validation changes only which rows travel together: every row is still predicted exactly once, by a model that still trained on about the same number of rows, and all 29 failures still count towards the score, each in the AUC of its own fold. There is far less left for the seed to move.

A predictable mistake is to take the mean of the five fold AUCs for one AUC computed on all

800 out-of-fold predictions pooled. It is natural — every drive is predicted exactly once — but they are different calculations: the pooled AUC also ranks drives scored by different models against each other, so it depends on whether the five models score on the same scale. On this fleet they do, and the two nearly coincide: 0.9519 pooled (`cross_val_predict`, seed 0) against the mean 0.9514. With less similar models they drift apart.

What 0.0032 is not: the error bar of 0.9514. It is how far cross-validation moves when only the fold cut changes on the same 800 drives. On new data it moves by 0.0165 (section 4.3). Cross-validation is less arbitrary, not precise to ± 0.003 — and reading it that way is **a predictable mistake**.

What it costs is k fits instead of one. For this course that is seconds; for a large network it is a real decision, and the usual compromise is a single validation set large enough that its own error bar is tolerable.

What it does not fix is leakage. Cross-validation measures whatever procedure it is handed, using all the data. If a step was fitted on every row before the folds were cut, every fold is contaminated the same way, and the folds agree with one another about a wrong answer — which looks exactly like a trustworthy result. Section 7 shows it happening.

---

5. What the error is made of

5.1 The picture first

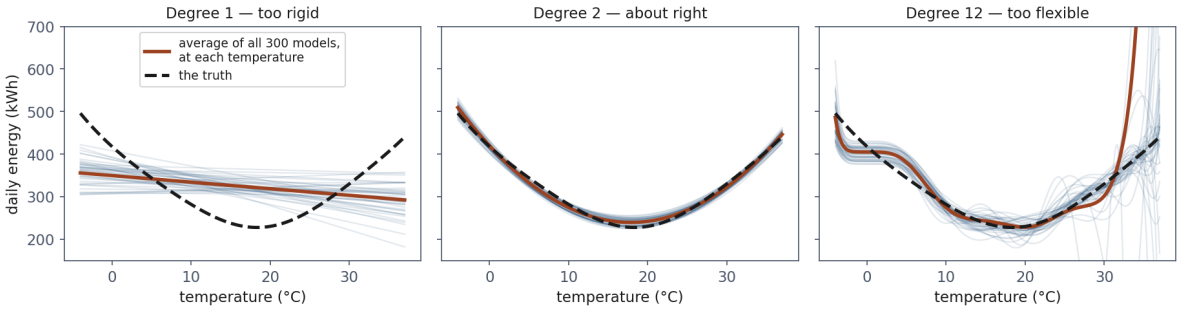
The question. Your model's validation error is too high. You can spend the next month collecting more data or changing the model, and the two cure different kinds of error: buy the wrong one and the month changes nothing. A straight line fitted to 25 days of the energy data below has a 5-fold cross-validated mean squared error of 6,064.3 kWh² — each round fits it on 20 days and scores it on the other 5, and 6,064.3 is the mean of the five — so a typical day is off by $\sqrt{6{,}064.3} \approx 78$ kWh. Section 5.5 says which cure it needs.

What we are after. A model trained on some data is asked about a new case, and its error is measured squared, as lesson 3's mean squared error. This section splits that squared error into three parts, two that belong to the model and one that belongs to the data, and then measures each with numbers. It does so in a lab: data we generated ourselves, so the true curve f and the noise σ are known. On real data the target y is measured but f and σ are not, and you have one training set, not three hundred. Section 5.5 brings the diagnosis back to real data, and section 6 adds the training-set size.

Why the lab, if real data cannot use the formula? Cross-validation gives two numbers per model; the decomposition is what says what they mean. The formula is the *why*, cross-validation the *how*. Only the formula explains why training error tracks bias and the gap tracks variance, why more data cures variance and not bias, why there is a floor below which a test error means leakage, and why the best model depends on the amount of data. The decomposition is old in statistics; Geman, Bienenstock and Doursat brought it into machine learning in 1992, in "Neural networks and the bias/variance dilemma".

Imagine drawing many different training sets from the same source and fitting the same model to each. You get many models, and they fail in two distinguishable ways.

Each faint line is one model, fitted to its own 25 days (40 of the 300 drawn). Bias: the red average against the black truth. Variance: how widely the faint lines spread around the red average.



Each faint line is the same model fitted to a different training sample of 25 days; forty of the 300 are drawn, and the red average is over all 300. Left, every line misses the curvature in the same way, and the red average sits away from the black truth: that gap is bias. The lines also fan out, more than the middle panel's: that fan is variance, not bias (section 5.3 explains why). Right, they disagree wildly, worst at both edges of the temperature range, where all the training points lie on one side: that is variance. In real life you drew one of these lines and never saw the others.

They may all be wrong in the same way. A straight line fitted to a curve misses the curvature every time, whichever sample it gets. That is **bias**: being consistently wrong. Whether the lines also agree with *each other* is a separate question, and here, as section 5.3 measures, they do not.

A predictable mistake is to read bias off the fan of lines: "the lines diverge, so the bias is high". It is a natural reading, since both are error. But bias is about where the *average* line sits against the truth, and variance about how far the individual lines sit from that average. Degree 1's lines fan out, and that fan is its variance; its bias is the gap between the red and the black curves.

They may disagree wildly with each other. A twelfth-degree polynomial fitted to twenty-five points gives a different wild curve every time. That is **variance**: being unreliable, whatever the average may look like.

It is worst at the two ends of the plot, for two reasons. At an edge every training point lies on one side, so nothing holds the polynomial down on the other. And the ends are partly extrapolation: the 25 temperatures are drawn between -5 and 38 °C, and the expected minimum of m uniform draws sits $1/(m+1)$ of the range above the bottom, so the coldest day is on average $-5 + 43/26 = -3.3$ °C and the warmest $38 - 43/26 = 36.3$ °C, while the plot runs from -4 to 37 °C. In more than half the training sets the coldest day is warmer than -4 °C: $(42/43)^{25} = 0.55$.

And underneath both sits the **noise** in the measurements, which no model removes because it is not a property of the model.

One word, two meanings. This *bias* is the bias of a **model**: too simple to follow the truth, wrong the same way on every training set. Sections 2 and 7 use the bias of an **estimate**: a measured score systematically too high or too low, as leakage makes it. **A predictable mistake** is to merge them — "cross- validation removes bias" — when it removes neither: it makes the estimate more precise, which is a third thing.

5.2 The bias–variance decomposition, derived

The result has a name worth knowing, because it is what to search for: the **bias–variance decomposition** of the expected squared error. It holds for squared error, the mean squared error (MSE)

of lesson 3; for accuracy or AUC there is no split this clean.

Let the data be generated as $y = f(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\mathrm{Var}(\varepsilon) = \sigma^2$. Fix a test point x, and write $\hat{f}$ for the model fitted to a random training set. The expectation below is over both the training set and the noise in the new observation y.

In the lab's terms: x is a temperature, say 30 °C; y is the **energy** measured on a new day at that temperature, in kWh — the truth $f(30) = 330.5$ plus fresh noise, so y is random too; $\hat{f}(x)$ is the prediction of **one** model, fitted to one random set of 25 days. **Two predictable mistakes**, both natural on a first read: taking y for the temperature, when the temperature is the input x; and taking $\hat{f}$ for the 300 models, as if one predicted with their average. In real life you train one model and predict with it. The 300 are averaged only to *measure* how wrong such a model is, and the average model $\bar{f}$ below exists only in a lab like this one, where it measures the bias.

A third predictable mistake, the most natural of all, is to write the error as $(f(x) - \hat{f}(x))^2$, true value minus prediction, because that is how everyone pictures an error. The left side has y, the energy *measured* on a new day, truth plus fresh noise, and that is exactly where σ^2 comes from: with $f(x)$ inside the square only bias² + variance would remain. The lab's formula fixes the distribution of y, not y itself — two new days at 30 °C give two different energies around 330.5 — which is why even a perfect model, $\hat{f} = f$, misses by $\sigma^2 = 484$ on average.

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \mathbb{E}\left[(f(x) + \varepsilon - \hat{f}(x))^2\right]$$

Expand that square, keeping ε apart from the rest, and three terms appear:

$$= \mathbb{E}\left[\varepsilon^2\right] + 2\,\mathbb{E}\left[\varepsilon\left(f(x) - \hat{f}(x)\right)\right] + \mathbb{E}\left[(f(x) - \hat{f}(x))^2\right]$$

The first is σ^2, by the definition of the noise. The middle one is zero, and it is worth seeing why rather than being told: ε is the noise in the *new* observation, while $\hat{f}$ was fitted on the training set, so the two are independent, the expectation of their product splits into the product of their expectations, and one of those factors is $\mathbb{E}[\varepsilon] = 0$. What survives is

$$= \sigma^2 + \mathbb{E}\left[(f(x) - \hat{f}(x))^2\right]$$

Now add and subtract $\bar{f}(x) = \mathbb{E}[\hat{f}(x)]$, the average model:

$$\mathbb{E}\left[(f - \bar{f} + \bar{f} - \hat{f})^2\right] = (f - \bar{f})^2 + \mathbb{E}\left[(\bar{f} - \hat{f})^2\right] + 2(f - \bar{f})\,\mathbb{E}\left[\bar{f} - \hat{f}\right]$$

The last term is zero, because $\mathbb{E}[\hat{f}] = \bar{f}$ by definition. What remains is

$$\boxed{\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{(f(x) - \bar{f}(x))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}\left[(\hat{f}(x) - \bar{f}(x))^2\right]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}}$$

Why the bias appears squared. The bias itself, $\bar{f}(x) - f(x)$, is a gap with a sign, in the units of the target: kWh here, and negative where the average model sits below the truth. Nobody squares it by choice: expanding the average squared error, as above, produces the term $(f(x) - \bar{f}(x))^2$ by

itself. Squared, it is in kWh², the units of the variance and of σ^2, which is what lets the three terms be added. A bias of +26.9 kWh and one of −26.9 kWh cost the same.

This is an identity, not an approximation: the two sides are equal exactly. The checks at the end of this section confirm it on the numbers.

In numbers, at 30 °C. The derivation fixed one test point x, so the example does too. Take the 300 straight lines (degree 1) of section 5.1, each fitted to its own 25 days, and ask every one of them for the energy at $x = 30$ °C. Notebook 2, section 2, prints each step:

Energies are in kWh; the three terms and their sum in kWh².

Step	Formula	Degree 1	Degree 2
The truth	$f(30)$, known because we wrote f	330.5	330.5
The 300 predictions	one per model	300.8, 317.1, 304.8, … from 215.6 to 364.7	334.1, 323.4, 318.5, … from 299.1 to 346.5
Their average $\bar{f}(30)$	sum ÷ 300	303.6	323.1
bias²	$(\bar{f}(30) - f(30))^2$	$(303.6 - 330.5)^2 = 723.9$	$(323.1 - 330.5)^2 = 53.7$
Variance	sum of $(\hat{f}(30) - \bar{f}(30))^2 \div 300$	856.1	58.4
Noise	$\sigma^2 = 22^2$	484.0	484.0
Expected error	bias² + variance + noise	2,063.9	596.1

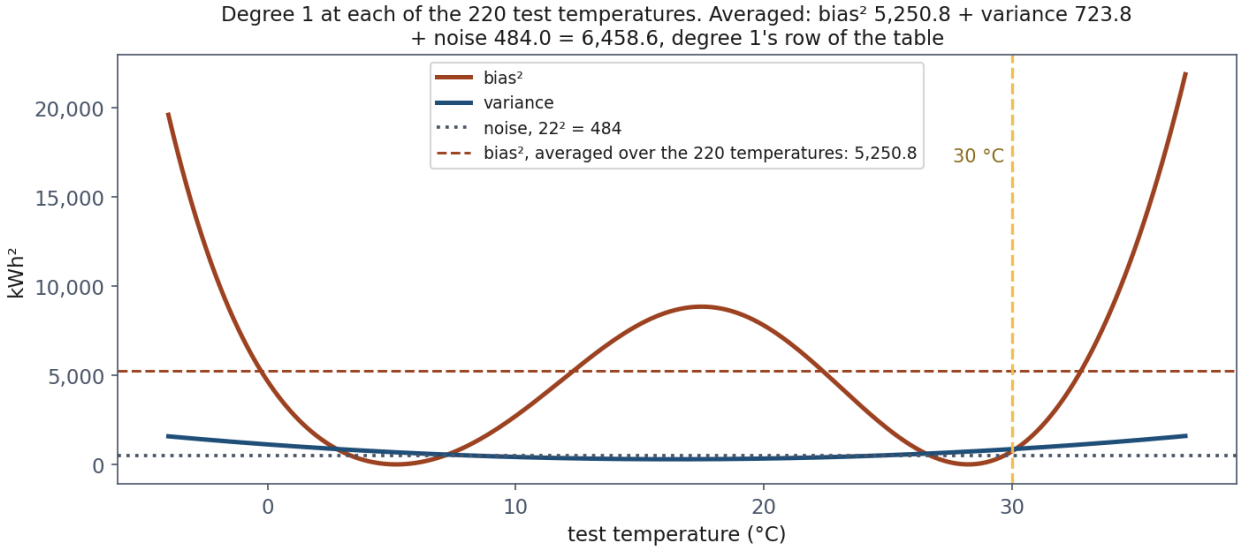
Read the degree-1 column as one sentence: a straight line trained on 25 random days, asked about a new day at 30 °C, misses by a squared error of about 2,064 kWh² on average — 724 because it cannot bend, 856 because it depends on which 25 days it saw, 484 because the new day is itself noisy. Back in kWh, $\sqrt{2{,}063.9} \approx 45$ kWh is the typical miss on such a day. Degree 2, the middle panel of section 5.1's figure and the best model of section 5.3's table, misses by 596, three and a half times less. The noise is not estimated from the models: we know it because we added it, and on real data it cannot be separated out. The slides show degree 1 only, rounded to whole numbers so that the column adds up: 724 + 856 + 484 = 2,064.

A predictable mistake in the variance row is to square after summing, $\left(\sum \hat{f}(30) - \bar{f}(30)\right)^2 \div 300$. The square goes on each term first; summed first, the differences from the average cancel to zero.

Checks, for whoever wants to recompute. Three, none needed to follow the argument.

- *Rounding.* Every value above is computed unrounded, so the printed ones can be off in the last digit: $(-26.9)^2$ gives 723.6, because the unrounded average and truth are 303.55 and 330.46; and degree 1's printed parts add to 2,064.0 against a total of 2,063.9.
- *Without the formula.* Give each of the 300 models 2,000 new days at 30 °C, truth plus fresh noise, and average the 600,000 squared errors: 2,061.0 for degree 1 and 594.2 for degree 2, within sampling error of the formula's 2,063.9 and 596.1.
- *Over all temperatures.* Notebook 2, section 2, computes both sides at the 220 test temperatures for six degrees; they agree to within 3×10^{-12}, which is floating-point equality.

Then at every temperature, and the average. Section 5.3's table repeats this at each of 220 test temperatures and averages them:



Degree 1, the same 300 models at each test temperature. Look at the red curve: bias² is zero where the straight line crosses the true curve, near 5 and 28 °C, and large at both ends and in the middle, so its average, the dashed line at 5,250.8, is far above its value at 30 °C. Variance, in blue, rises toward the ends, where the line pivots; it averages 723.8. With the noise, 484, the three averages make the first row of section 5.3's table: 6,458.6.

5.3 What it looks like on real numbers

Energy against outdoor temperature, one observation per day, from the curve `true_curve` in `Notebooks/methodology_data.py`:

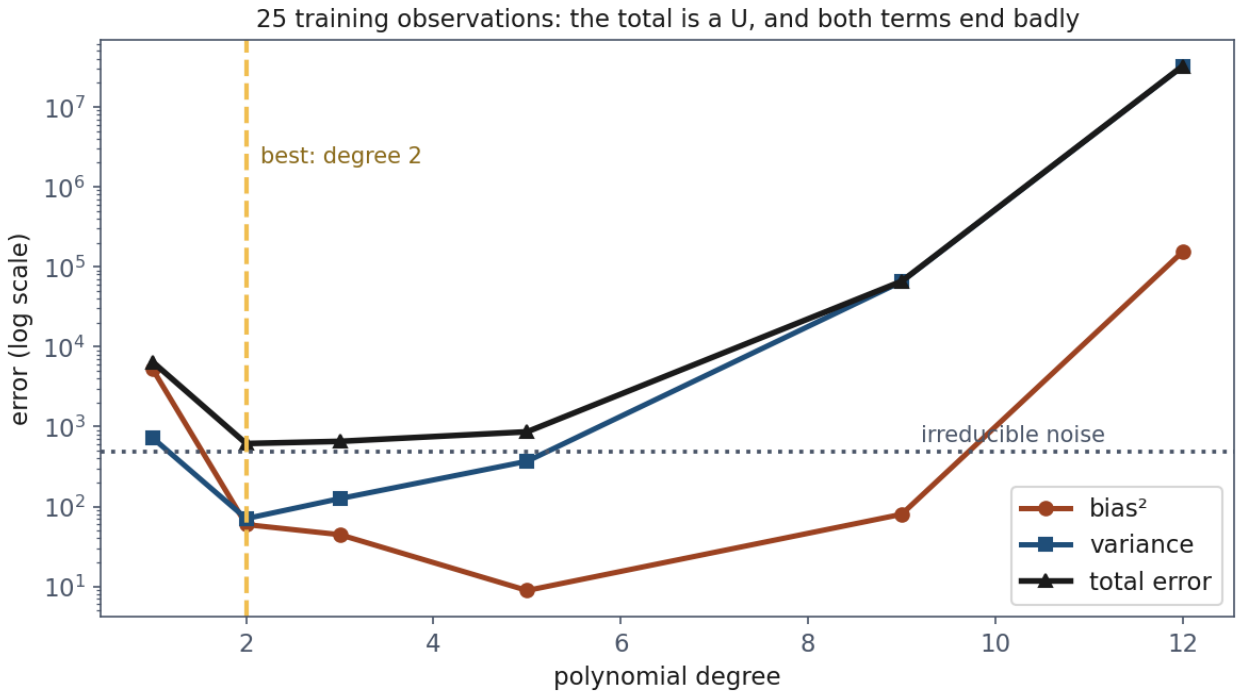
$$f(t) = 240 + 0.55\,(t - 18)^2 + 12\sin(t/4)\ \text{kWh}$$

Here t is the day's mean outdoor temperature; the 18 is the balance temperature of the degree days energy utilities use, below which a building heats and above which it cools. Each training set is 25 days scattered over the year, not consecutive, at mean temperatures drawn uniformly between −5 and 38 °C, with energy $f(t)$ plus noise of standard deviation 22 kWh, so a noise variance of $22^2 = 484$. The uniform draw is not a real year, where extreme days are rare, and a daily mean of 38 °C is extreme: the range is chosen to cover the curvature and the edges. There are 300 training sets, each a **new, independent sample** drawn afresh from the curve; the 25 days of section 5.1's question are the first of them. **A predictable mistake** is to read them as 300 splits of one sample, as cross-validation makes; splits share most of their data, so models trained on them agree too much and the variance would come out too small. Every number below is averaged over the 300 training sets and over 220 test temperatures from −4 to 37 °C. The 220 are an evenly spaced grid, one every 0.19 °C: the error depends on the temperature, so to get one number per degree the 30 °C computation of section 5.2 is repeated at each of them and averaged — in effect, an average over the whole range. Any fine grid gives nearly the same averages; −4 to 37 sits just inside the data's −5 to 38. The **same** 300 training sets serve every degree, so the degrees differ only in the model: 6 degrees × 300 = 1,800 models, each asked at 220 temperatures, 396,000 predictions. No new day is

simulated: bias² and variance need only the predictions and the truth, and the noise enters exactly, as 484.

A predictable mistake is to take this for lesson 3's dataset. The setting is the same, energy against temperature with noise of 22 kWh, and so is the shape. The curve is not: lesson 3 used a plain parabola, $240 + 1.15\,(t - 18)^2$, on 30 days between −5 and 35 °C. This one is gentler and carries a seasonal sine term, so no polynomial of any degree matches it exactly and every degree keeps some bias. We wrote it ourselves because measuring bias needs the truth, and the noise floor needs the noise level; real data gives neither.

This shape has a name of its own, the **bias–variance trade-off**: the consequence of the decomposition, as flexibility lowers one term and raises the other.



The total error is a U whose minimum, at degree 2, is the model you want. What to look at is the red curve after degree 5: bias² climbs four orders of magnitude, which is not what the textbook picture of bias leads you to expect. The prose below explains it, and the explanation is the point of the section. The scale is logarithmic — on a linear axis degree 12 would flatten everything else onto the baseline — and the dotted line is the noise floor at 484, which no model crosses. Only degrees 1, 2, 3, 5, 9 and 12 are measured; the segments between them only join the points.

Degree	bias² (kWh²)	Variance (kWh²)	Noise (kWh²)	Total = bias² + variance + noise (kWh²)
1	5,250.8	723.8	484.0	6,458.6
2	59.6	70.8	484.0	**614.3**
3	44.5	125.8	484.0	654.4
5	8.9	368.9	484.0	861.8
9	80.1	65,186.0	484.0	65,750.1

Degree	bias² (kWh²)	Variance (kWh²)	Noise (kWh²)	Total = bias² + variance + noise (kWh²)
12	153,537.3	32,174,344.9	484.0	32,328,366.2

The totals add the unrounded parts, so the printed ones can differ in the last digit: $59.6 + 70.8 + 484.0 = 614.4$, printed 614.3.

Degree 1 is a bias problem — its bias² is 5,250.8, two orders of magnitude above the next row, and its total error is the third worst in the table. Wrong everywhere, and in the same way on every sample.

Its variance is not the smallest, though, and that is worth a minute. The textbook picture has variance rising with complexity from the very first point; here degree 1 measures 723.8 against degree 2's 70.8. Nothing is broken. Each of the 300 universes draws its own 25 temperatures, and a straight line *cannot* follow the curve, so which line it settles on depends on where those temperatures happened to fall: a sample weighted towards cold days tilts it one way, a sample weighted towards hot days the other. The line pivots from sample to sample even though it is the least flexible model in the table. Hold the 25 temperatures fixed across universes and the effect disappears: notebook 2 measures the same six models that way, and the variance column then climbs from the first row to the last — 33.9, 55.3, 81.9, 154.5, 709.2, 2,669.8 — with degree 1 the most stable model in the table, as the picture promises.

So the clean monotone curves of the textbook diagram are what a well-specified model does on a fixed design. Measured on a misspecified one, bias² falls only as far as degree 5 and then climbs, and variance starts high before it starts rising. What survives is the shape that matters: a total error that is a U, with its minimum at degree 2.

Degree 12 is a variance problem, and the magnitude is worth pausing on: 32 million, four orders of magnitude larger than anything else. On 25 points a degree-12 polynomial nearly interpolates, and an interpolating model is a machine for amplifying noise.

Its bias² also looks terrible, which seems to contradict the claim that flexible models have low bias. Bias and variance stay separate quantities; what fails here is the *measurement*. A few of the 300 degree-12 fits fly off at the edges of the range, where every training point lies on one side, and those few dominate both columns. The average of 300 such curves is itself unstable: an estimated bias² carries the uncertainty of that average, on average variance $\div$ 300 = 32,174,344.9 $\div$ 300 $\approx$ 107,000 kWh², the same size as the 153,537.3 measured. And another seed does not settle it: on seeds 1 to 5, bias² runs from 39,016 to 1,566,449 and variance from 2.8 million to 547 million. **Read the two numbers together**: degree 12 is unusable, and its exact digits mean nothing.

The noise column never moves. 484, at every complexity, because it is a property of the data. It is the floor: no model, no algorithm and no quantity of data brings the expected error below it.

That has a practical corollary worth stating plainly. **A test error below the noise floor is evidence of leakage or of a mistake in the evaluation, not of excellence.** If you know your measurement precision and your model beats it on data it never saw, something has leaked. The condition matters: the floor bounds the error on *new* data. A training error can go far below it — degree 12 on its own 25 points does, by fitting the noise.

Two more conditions make the check usable. **Well below, not just below**: the floor bounds the *expected* error, and a small test set can dip under it by luck. Even the true curve, scored on n new days, has a mean squared error equal to the mean of n squared noises, $\sigma^2\chi^2_n/n$. On 5 test days it falls below half the floor, 242, with probability 0.22 — about one time in five; on 50 days, with probability 0.0012. **Known from outside**: here 484 is known because we added it; on real data the floor comes from the instrument's stated precision or from repeated measurements under the same conditions, and without it the check cannot be made.

5.4 The optimum depends on how much data you have

The same experiment at three training-set sizes, showing total expected error:

Degree; total error at training-set size m	m = 25	m = 60	m = 150
1	6,458.6	5,998.6	5,861.5
2	**614.3**	570.6	555.3
3	654.4	564.6	544.0
5	861.8	**540.9**	**506.9**
9	65,750.1	756.6	514.0
12	32,328,366.2	7,626.9	899.0

The best degree moves from **2 to 5** as the sample grows. Degree 12 improves by a factor of $32{,}328{,}366.2 \div 7{,}626.9 = 4{,}239$ between 25 and 60 observations and of $32{,}328{,}366.2 \div 899.0 = 35{,}960$ by 150. It is the worst model at 25 and at 60; at 150 it has overtaken the straight line, whose 5,861.5 no quantity of data repairs — bias does not shrink with m.

> **"Which model is best?" is not a question about models. It is a question about models *and* the quantity of data you have.**

This explains a pattern you will meet constantly: a paper reports that a large model beat a small one on a dataset far bigger than yours, and the result does not reproduce for you. Neither experiment was wrong.

5.5 Without the truth: training error against validation error

Everything above needed f, σ and 300 training sets. Real life gives one training set and the measured y. That is enough for two errors per model, computed by cross-validation **on the training data, with the test set untouched**:

- **Training error**: each round's model scored on the days it was fitted to. It has already bent towards those days, noise included, so the spread that variance measures does not show; what it reflects is, roughly, bias plus the noise the model did not chase.
- **Validation error**: the same model on the fold it did not see. That is a new day for it, so it pays bias, variance and noise.

So a training error too high for the task points to bias, and a validation error far above the training error points to variance. The rule is the decomposition put to work.

A predictable mistake is to read a low training error as low variance. The training error *hides* variance, it does not measure it: the model has bent towards its own days, so how differently it would

have come out on other days never shows. Degree 12 has a tiny training error and an enormous variance. What is unknown is the *true rule* f, not our model, which we choose.

High against what? In the lab the reference was the noise, 484; on real data σ is unknown, so the reference comes from outside. First, the error the task needs. The root of the mean squared error, the root mean squared error (RMSE), is in kWh, the typical daily miss: the straight line below trains at $\sqrt{5{,}155.7} = 72$ kWh and validates at $\sqrt{6{,}064.3} = 78$ kWh. If a building manager needs forecasts within about 50 kWh to buy energy ahead, the line already misses by more on the days it has *seen*, the easiest test there is, so it cannot do better on new ones: it is too rigid, and more days will not change that. Degree 2 trains at $\sqrt{312.8} = 18$ and validates at $\sqrt{450.4} = 21$ kWh, both within 50. Second, the instrument's known precision, when there is one. Third, a more flexible model: if degree 2 trains at 312.8 where the line stays at 5,155.7, the line cannot follow the data. And the two problems can come together, a high training error *and* a validation error far above it; pure bias is a high training error with the validation error close to it.

One sample of 25 days from the curve of section 5.3 (seed 0), 5-fold cross-validation (`KFold(n_splits=5, shuffle=True, random_state=0)`), so each round fits on 20 days and scores the other 5; each number is the average over the five rounds, from notebook 2, section 4. In code: `cross_validate(model, X, y, cv=folds, scoring="neg_mean_squared_error", return_train_score=True)`.

Degree	Training MSE	Validation MSE	Gap = validation − training	Reading
1	5,155.7	6,064.3	908.5	both high and close: bias
2	312.8	**450.4**	137.7	lowest validation: the best
3	306.1	525.6	219.5	
5	262.4	615.1	352.6	training below 484: variance

The gaps are computed from the unrounded errors, so $615.1 - 262.4 = 352.7$ is printed 352.6.

Read it against the lab. **Degree 1** is as wrong on the days it has seen as on the days it has not: bias, and the lab measured its bias^2 at 5,250.8. **Degree 2** has the lowest validation error, as it had the lowest total in the lab. **A predictable mistake** is to pick the model with the lowest *training* error, which looks like the most accurate: it would pick degree 5, 262.4 against degree 2's 312.8, the one chasing the noise. The choice is made on the validation error. **Degree 5** has a training error of 262.4, *below* the noise variance of 484: no model predicts new days better than the noise allows, so a training error below it means the model has fitted part of the noise. Its gap is the widest of the four. Degree 12 is left out: with unscaled powers up to t^{12} the least-squares solver loses precision, and its training error is not trustworthy.

The numbers differ from the lab's, and must: one sample and five folds of 20 days against an average over 300 samples. What carries over is the ordering, and a diagnosis needs nothing more.

The answer to the question of section 5.1. The straight line's error is bias, so more days will not cure it: in the lab, six times the data takes it only from 6,458.6 to 5,861.5 (section 5.4). Changing the model to degree 2 brings the validation error from 6,064.3 to 450.4 on the same 25 days.

With a sample of 150 days the same table reads:

Degree	Training MSE	Validation MSE	Gap
1	7,281.1	8,058.6	777.5
2	593.5	627.0	33.5
3	584.2	628.4	44.2
5	537.8	**596.0**	58.3

Degree 5's gap falls from 352.6 to 58.3 and it now has the lowest validation error, as the lab found at 150 days: more data cured its variance. Degree 1 stays high. That it comes out *higher* than on 25 days is the sample, not the size: a different draw of 150 days, where the lab's average over 300 draws falls.

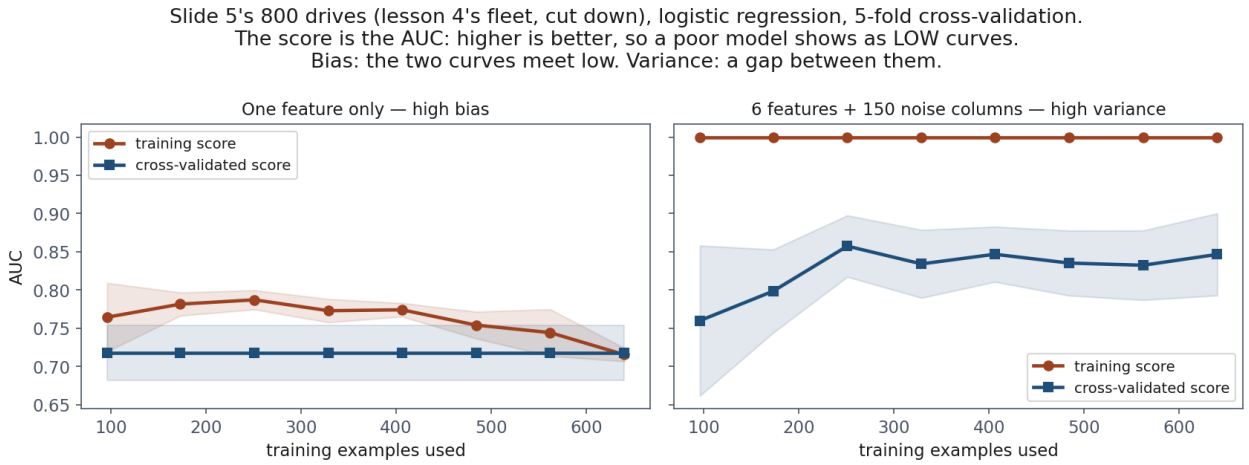
A predictable mistake is to conclude that, since f is unknown on real data, the decomposition is only a proof with no use. It is a reasonable reading — none of the lab's columns can be computed on real data. But the decomposition is what gives the two measurable errors their meaning, and what says which cure each one needs.

---

6. Learning curves

Section 5.5 measured the training and the validation error at one training-set size. A learning curve measures them at several sizes and plots both, which adds what the question of section 5.1 needs: whether more data would help.

The example switches to section 2's 800 drives — lesson 4's fleet, cut down — scored by the area under the curve (AUC). The method is the same for any score, but the direction flips: an error is better low, an AUC is better high, so here a poor model shows as *low* curves.



Two diagnoses, on section 2's 800 drives (lesson 4's fleet, cut down); the score is the AUC, so higher is better and a poor model shows as low curves. Left: the curves meet, and meet low — bias, and more data will not help; the validation curve is exactly flat, for the reason given below the table. Right: a gap of 0.15 that does not close — variance. Read the right-hand curve to the end, though, rather than stopping at its rise: it climbs to 0.857 at 251 examples and then moves

sideways, finishing below its own high point at 0.847. Both halves are information, and Section 6.1 says what each one licenses.

The construction. 5-fold cross-validation on the 800 drives, so each round's training part holds 640 and the validation fold 160. The curve trains on a growing share of the 640 and always scores the same 160 held-out drives, so each model is refitted 8 sizes × 5 rounds = 40 times. The demo uses all 800 drives because nothing from it is reported as a final score; in practice, set the test set aside first and run the learning curve on the rest. Train on 15% of it, 96 drives, then on eight evenly spaced sizes up to all 640, plotting both the training score and the cross-validated score against the number of examples used. The *shapes* carry the diagnosis.

A predictable mistake is to read the rightmost point as the model you will deploy, trained on all 800 drives. It is the largest training part a round holds, 640 of the 800; the deployed model is refitted afterwards on all the development data, and no point of the curve describes it exactly.

Measured on the fleet, with two deliberately broken logistic regressions:

Model	Training	Validation	Gap	Validation trend
`power_on_hours` only, C = 1	0.715	0.718	−0.002	0.718 at every size: exactly flat
6 features + 150 noise columns, C = 1000	**1.000**	0.847	+0.153	from 0.760, up to 0.857 at 251, then level

Why the left curve is exactly flat. With a single feature, logistic regression ranks the drives in that feature's own order whatever coefficient it learns, so its AUC is the AUC of `power_on_hours` itself, 0.718, and no amount of training data can move it. It is the extreme case of bias, chosen to make the shape unmistakable; a real high-bias curve rises a little at first and then flattens low.

6.1 Reading them

High bias. The curves meet, and they meet *low*. **A predictable mistake** is to read bias off the meeting alone: a good model's curves also approach each other as data grows, but high, near 1. The word that carries the diagnosis is *low* — here 0.715 against 0.718. The model is equally mediocre on data it has seen and data it has not, because it has extracted everything available to it and that is not enough — on the left, literally everything one column's ordering contains.

- *Helps:* a more flexible model, better features, less regularisation.
- *Does not help:* **more data.** The curve is already flat; more rows land on the same plateau.

High variance. A wide gap between a high training score and a lower validation score. The gap is the diagnosis; what to do about it is written in the *slope* of the validation curve, and the two must be read separately.

- **Still rising at the right edge, by more than the fold-to-fold spread?** Then more data is the likely remedy, and the rising curve is the evidence that it should pay. This is the only case in which "collect more" is a defensible plan rather than a hope.
- **Risen and then flattened**, which is what the figure above actually shows — 0.76 to 0.86 by 250 examples, and level from there? Then the gap is real but more rows of the same kind have stopped buying anything. The remedies left are the other ones: a stronger penalty (lesson 3's

λ; in `LogisticRegression` a smaller `C`), removing features, a simpler model. In that figure the 150 noise columns are the problem, and no quantity of rows removes a column. "Removing features" means dropping columns; **a predictable mistake** is to hear "drop" and think of dropout, a neural-network technique (lesson 9) that has nothing to do with it.

The practical point, and it is worth stating separately: the plot tells you whether the expensive remedy is worth buying *before* you buy it. Collecting data is the slowest and costliest item on the list, and a flat curve says in advance that it will change nothing.

6.2 The one-line diagnostic

> **Is the gap large, and is the validation curve still rising at the right edge, by more than its spread?** Both together are the only case in which more data is the answer. A curve that has levelled off says more rows buy nothing — whether it levelled off with the curves together (bias) or far apart (variance, which is what our own figure shows).

And measure the slope against the band before believing it. The right-hand curve ends on a segment that rises by 0.014, which the eye reads as a curve still climbing. Across the five rounds that step is -0.030, $+0.050$, $+0.006$, -0.009, $+0.053$: an average of $+0.014$ with a standard deviation of 0.036. The curve's highest point is at 251 examples, 0.857, and it finishes *below* it at 0.847. A slope smaller than the band drawn around it is not a slope.

The predictable mistake here is prescribing data for a bias problem, and the instinct behind it is entirely sound — more data almost always helps, it is the one intervention that never makes a model worse, and it is what every textbook recommends. It simply does not help *this* failure, and a learning curve is fifteen lines of code that tells you which failure you have.

And the test set stays out of the diagnosis. Every number in sections 5.5 and 6 comes from cross-validation inside the training data; the test set is scored once, after the fix is chosen. **A predictable mistake** is to diagnose on the test error — it is the number everyone looks at, so the instinct is natural. But if the final test score disappoints and you change the model because of it, the test set has become a validation set (section 8: train to fit, validation to choose, test to report once), and its number is no longer an honest estimate.

---

7. Leakage that survives cross-validation

Lesson 2 covered leakage in preprocessing. This section is about the leakage cross-validation does **not** catch, which is more dangerous precisely because you believe you are protected.

7.1 The catastrophe

Build a table of **2,000 columns of pure random noise**, independent of the label and of each other. The honest performance of any model on it is 0.500.

Now write the code almost everyone writes:

```
selector = SelectKBest(f_classif, k=10).fit(noise, y)   # all the data
chosen = noise.loc[:, selector.get_support()]
cross_val_score(model, chosen, y, cv=folds, scoring="roc_auc")
```

What the F-test asks, in words. `f_classif` takes one column at a time and asks: in this column, do the failing drives and the healthy ones have different means? It answers with a ratio — how far apart the two groups' means are, divided by how widely the values scatter inside each group. Formally that is the between-class variance over the within-class variance, an analysis-of-variance (ANOVA) F statistic; with two classes it is the square of the familiar t statistic. A large value says the column seems to separate the classes; `SelectKBest` computes it for all 2,000 columns and keeps the ten largest.

Why pure noise still offers ten columns that look good. The mean of 29 standard-normal values is not exactly 0: it wobbles with standard deviation $1/\sqrt{29} = 0.19$. Across 2,000 columns, a few drift far by luck — only 6 of the 2,000 by more than three of those standard deviations. The ten the selector keeps have the failing drives' mean shifted by 0.52 to 0.77 from the healthy drives', between 2.8 and 4.1 times what chance usually allows. They look like signal, and they are the luck of those 29 drives.

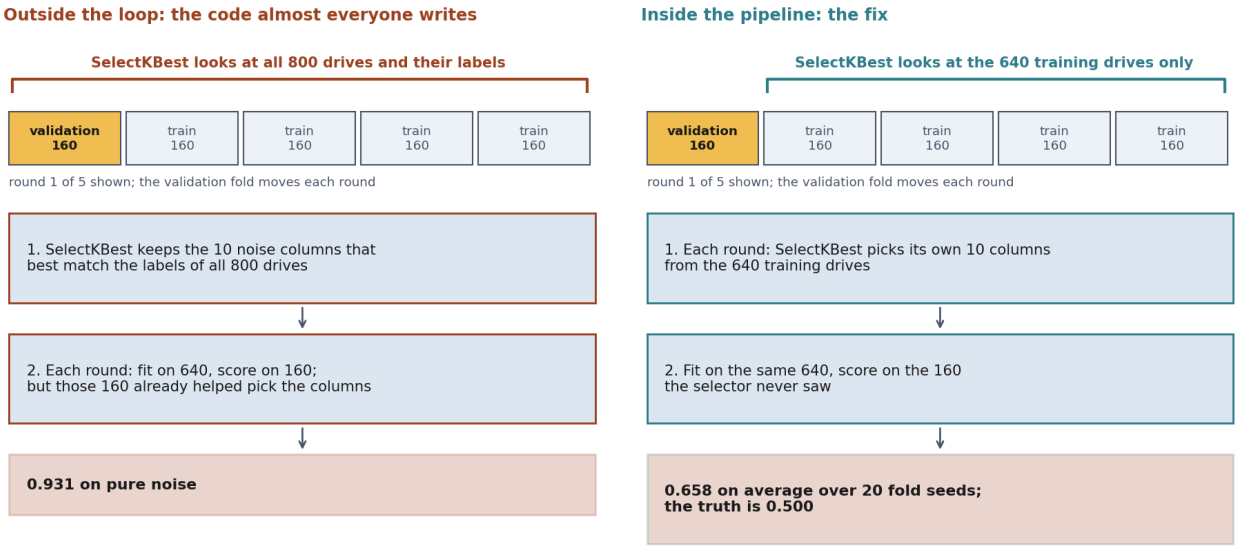
Cross-validated AUC: 0.931, the mean of five folds: $(0.928+0.979+0.975+0.943+0.832) \div 5 = 4.657 \div 5 = 0.931$. Four of the five land between 0.92 and 0.98 — which is exactly what a trustworthy result looks like.

The error is on the first line. `SelectKBest` was shown every row, including those that would later serve as validation folds. It searched 2,000 columns for the ones that best matched labels it had already seen, and handed the winners to cross-validation.

Cross-validation did not fail. It was lied to. It faithfully measured a procedure that had already peeked.

7.2 The fix, and the surprise

Put the selection inside the pipeline so it is refitted in every round, on training rows only.



The bracket marks the drives `SelectKBest` looks at. Left, section 7.1's code: all 800, the validation fold included, so the drives being scored helped pick the columns. Right, the selector as the pipeline's first step: only the 640 training drives of each round.

Over 20 different fold seeds the estimate becomes **0.658** — mean of 20, ranging 0.576 to 0.759.

Better. **And not 0.500.**

This is not a bug, and understanding why is the most useful idea in the lesson.

With 2,000 columns and 800 rows, some columns correlate with the label by accident across the whole dataset. That accident is a property of this particular sample. It is present in every subset of it — in each training fold and in each validation fold alike. A selector fitted honestly on four fifths of the data finds those columns, and they still "work" on the remaining fifth, because the spurious correlation was never fold-specific.

> Cross-validation protects you from leaking **between folds**. It cannot protect you from having searched a large space of possibilities on a small sample. No rearrangement of the same 800 rows removes a pattern that is genuinely present in those 800 rows and absent from the world.

What does help — three things, none of them another cross-validation, since cross-validation reshuffles the same 800 rows, and the accident lives in them:

- **A test set held out before any of this began, and used exactly once.** The chosen columns' chance match lives in the rows the search saw; in new rows it is a fresh coin toss. The paragraph below takes up the obvious objection.
- **Fewer candidates.** The luckiest of 2,000 columns is luckier than the luckiest of 20, as the best of 2,000 lottery tickets beats the best of 20. The fewer columns, models or settings tried, the less the winner is inflated by chance.
- **More rows.** What matters is the count in the rarer class: a noise column's mean over 29 failures wobbles by $1/\sqrt{29} = 0.19$, over 290 by $1/\sqrt{290} = 0.06$. With more rows the chance matches shrink, and the selector finds nothing convincing.

But a fresh test set has chance correlations too. It does, in other columns, which the model does not use. The model uses the columns chosen because they matched the labels of the 800 training drives; in new rows their correlation with the label is a fresh coin toss, positive or negative, zero on average. So a fresh test set is centred on the truth, though one of them can still land high by luck.

Two kinds of error, and each cure removes one. The search's inflation is *systematic*: it pushes the score up, and no cross-validation removes it, because the accident is in every fold of the same sample. A fresh test set's error is *random*: the score wobbles, because a few hundred drives hold only a handful of failures. A test set set aside before any choice trades the first for the second — unbiased, not precise — which is why a report says how many failures it held.

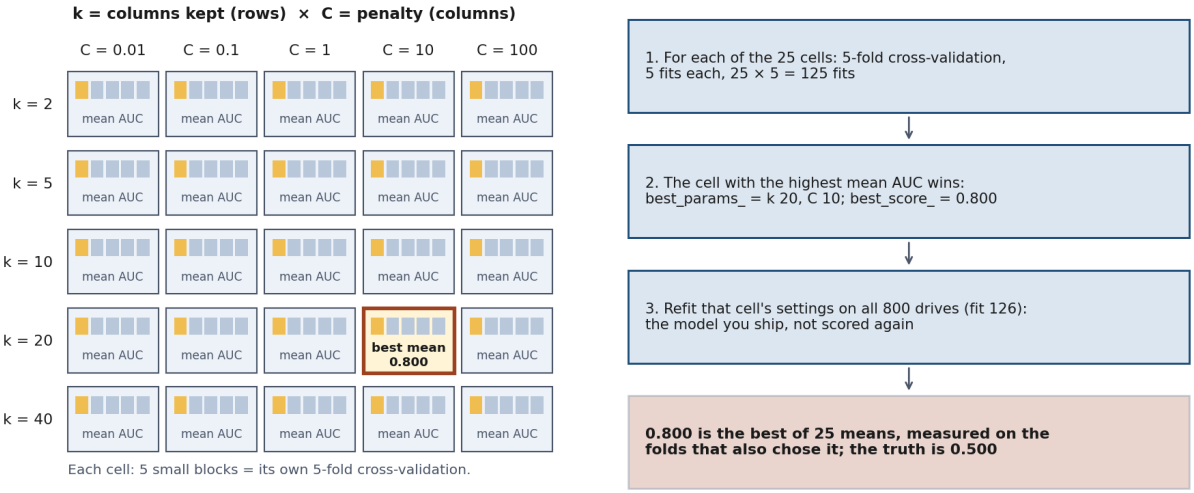
Checks, for whoever wants the detail. Simulated: the honest pipeline ($k = 10$) and the search's winner of section 7.3, both fitted on notebook 3's 800 noise rows, scored on 40 fresh sets of 800 drives — new labels from the fleet generator, new noise columns. The honest pipeline averages an AUC of 0.509, from 0.40 to 0.68; the search's winner averages 0.512. Centred on 0.5, as the truth requires, and noisy, which is why a report states how many positives its test set held.

7.3 Hyperparameter search is the same problem

Section 3 chose between three values of one hyperparameter. A real search chooses between dozens, over several hyperparameters at once — and choosing is choosing, whatever it is called, so it costs the same honesty.

Section 7.2's honest pipeline still has two settings someone must choose: k, the number of columns `SelectKBest` keeps, and C, the setting of the logistic regression's penalty (smaller C, stronger penalty). `GridSearchCV` is the loop that chooses them, in four steps:

1. **A grid.** $k \in \{2, 5, 10, 20, 40\}$ and $C \in \{0.01, 0.1, 1, 10, 100\}$: 5 values of k times 5 values of C, $5 \times 5 = 25$ combinations.
2. **Each combination scored by 5-fold cross-validation** with the honest pipeline: $25 \times 5 = 125$ fits.
3. **The highest mean AUC wins.** `best_params_` names it ($k = 20$, $C = 10$), `best_score_` is its mean AUC.
4. **The winner is refitted on all the data**; `best_estimator_` is that model.



The four steps as a grid: rows are k, columns are C, and each cell holds its own 5-fold cross-validation. The outlined cell has the highest mean; its settings are refitted on all 800 drives and not scored again.

The order matters: `best_score_` is computed in step 3, on the folds, before the refit. The refitted model is the one you ship, and it is not scored: scored on the 800 drives it trained on, it would give a training score.

In notebook 3 it is one line, `GridSearchCV(honest, grid, cv=inner, scoring="roc_auc").fit(noise, y)`, where `grid` has the keys `"select__k"` and `"model__C"` (the step's name, two underscores, the parameter's name) and `inner` is `StratifiedKFold(5, shuffle=True, random_state=1)`.

What `best_score_` is made of. Each combination runs its own 5-fold cross-validation, so it gets five AUCs, one per round — a fit on 640 drives, a score on the other 160 — and their mean. Twenty-five rows like these, of which four are shown (notebook 3 prints the winner's):

Combination	Round 1	Round 2	Round 3	Round 4	Round 5	Mean = sum ÷ 5
$k = 20$, $C = 10$, the winner	0.750	0.754	0.879	0.740	0.877	\$4.000 ÷5 = \$ **0.800**

Combination	Round 1	Round 2	Round 3	Round 4	Round 5	Mean = sum ÷ 5
$k = 20$, $C = 1$, second	0.732	0.748	0.899	0.728	0.884	0.798
$k = 20$, $C = 100$, third	0.751	0.741	0.876	0.744	0.879	0.798
$k = 2$, $C = 0.01$, last	0.717	0.641	0.712	0.610	0.739	0.684

`best_params_` is the row with the highest mean; `best_score_` is that mean, 0.800. Its five AUCs come from five models trained on 640 drives each, during the search; the model refitted on all 800 comes after, and nobody scores it. The top three are nearly tied: the winner wins by a hair, as in section 3.

A third predictable mistake: "25 values of C, times 5 columns, 125 fits". The 125 is right and the reasoning is not: five values of k times five of C make the 25 combinations, each costs five fits, and the refit is fit 126.

A predictable mistake: "k in the grid is the number of folds." The reading is natural — k has meant the folds since section 4 — but here it is the selector's parameter, the number of columns kept; the folds are fixed at 5 by `cv=inner`. **A second one: "`best_score_` is the AUC of the model refitted on all the drives."** Natural, since the refit is the last step listed, but the score was computed first, on the folds.

The 25 combinations on that signal-free table:

	AUC
best of 25 candidates (`best_score_`)	**0.7999**
average candidate	0.7265
worst candidate	0.6840
the luck of the winner: best − average	$0.7999 - 0.7265 = 0.0734$
the truth, by construction	0.5

Two inflations, one on top of the other. The *average* candidate, 0.7265, is already far above the truth: that is section 7.2's accident of the sample, present even with the selection inside the pipeline. The *best* adds its own luck on top. The number a practitioner reports is `best_score_`, and it is the **maximum of 25 noisy estimates**. The maximum of a set of noisy numbers is biased upward even when every one measures the same quantity. **0.7999 is not the performance of the chosen configuration; it is the performance of whichever configuration got luckiest.** How much choosing costs, measured honestly, is the subject of section 7.4.

7.4 Nested cross-validation

Measure the search, not the winner.

The bridge first: nested cross-validation is section 3's recipe, five times. Section 3 set a test set aside, chose a hyperparameter on the rest, refitted the winner and scored the test set once. Nested cross-validation does exactly that, with cross-validation as the way of choosing, and repeats it five times with a different fifth of the drives as the test set, so that every drive is tested once.

Ordinary cross-validation measures a *fixed* model, say $k = 10$ and $C = 1$: nothing is chosen while it measures, so its mean is honest. `GridSearchCV` gives the same folds two jobs at once — choose among 25 candidates, and score the winner — and the winner is whichever got luckiest on those very folds. It is a student allowed to pick which exam to sit after reading all the questions: the mark is real, and it says little about the next exam.

Nested cross-validation separates the two jobs. The whole search, choosing included, is treated as the model, and cross-validated like any other:

```
outer = StratifiedKFold(5, shuffle=True, random_state=2)
cross_val_score(GridSearchCV(honest, grid, cv=inner, scoring="roc_auc"),
                noise, y, cv=outer, scoring="roc_auc")
```

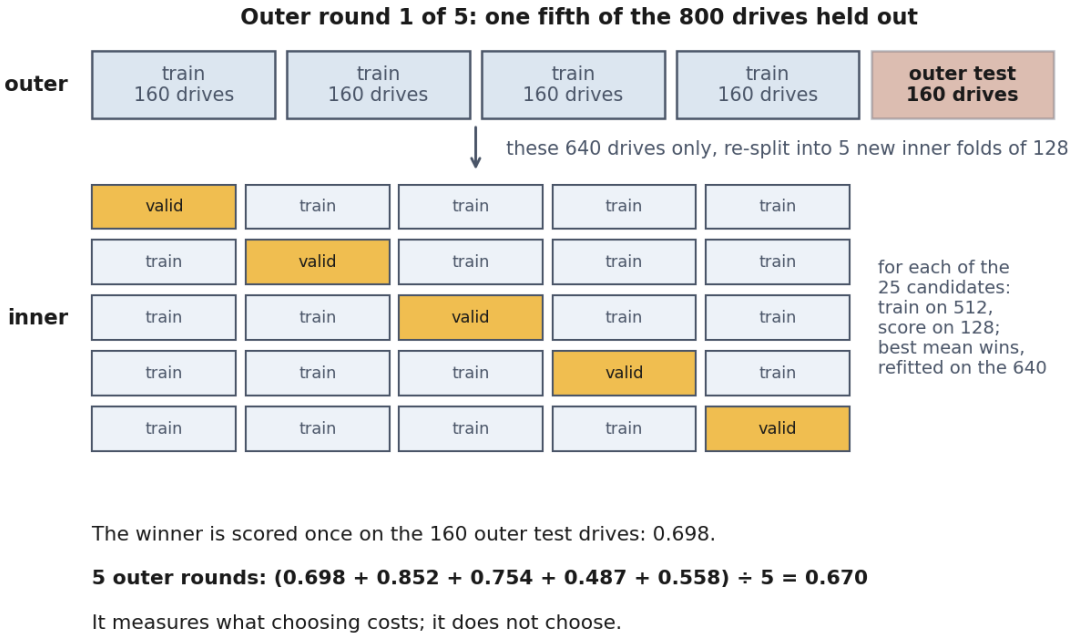
Outer round 1, step by step, with notebook 3's seeds (outer 2, inner 1):

1. **Set aside outer fold 1:** $800/5 = 160$ drives, about six failures. Nothing that follows sees them.
2. **The other 640 pass to the search as one block** and are re-split into five new inner folds of $640/5 = 128$; the inner folds are not the four outer blocks of 160. For each of the 25 candidates, five inner rounds: train on $640 \times 4/5 = 512$, score the AUC on 128; the mean of the five is that candidate's score. $25 \times 5 = 125$ fits.
3. **The best inner mean wins.** It is this search's own `best_score_`: a maximum, so optimistic on average as in section 7.3, though on inner folds of 128 drives with four or five failures the noise can hide it. It is thrown away.
4. **The winner is refitted on the 640:** fit 126.
5. **That one model is scored once on the 160 set aside:** AUC 0.698.

Rounds 2 to 5 repeat everything with another outer fold, and possibly another winner: 0.852, 0.754, 0.487 and 0.558. Every drive is in the outer test exactly once, and the whole procedure costs $5 \times 126 = 630$ fits. The nested figure is the mean of the five outer scores:

$$\frac{0.698 + 0.852 + 0.754 + 0.487 + 0.558}{5} = \frac{3.349}{5} = 0.670$$

(0.6699 unrounded). The inner loop **chooses**, and its scores are thrown away; the outer loop only **measures**, on drives that took no part in the choice.



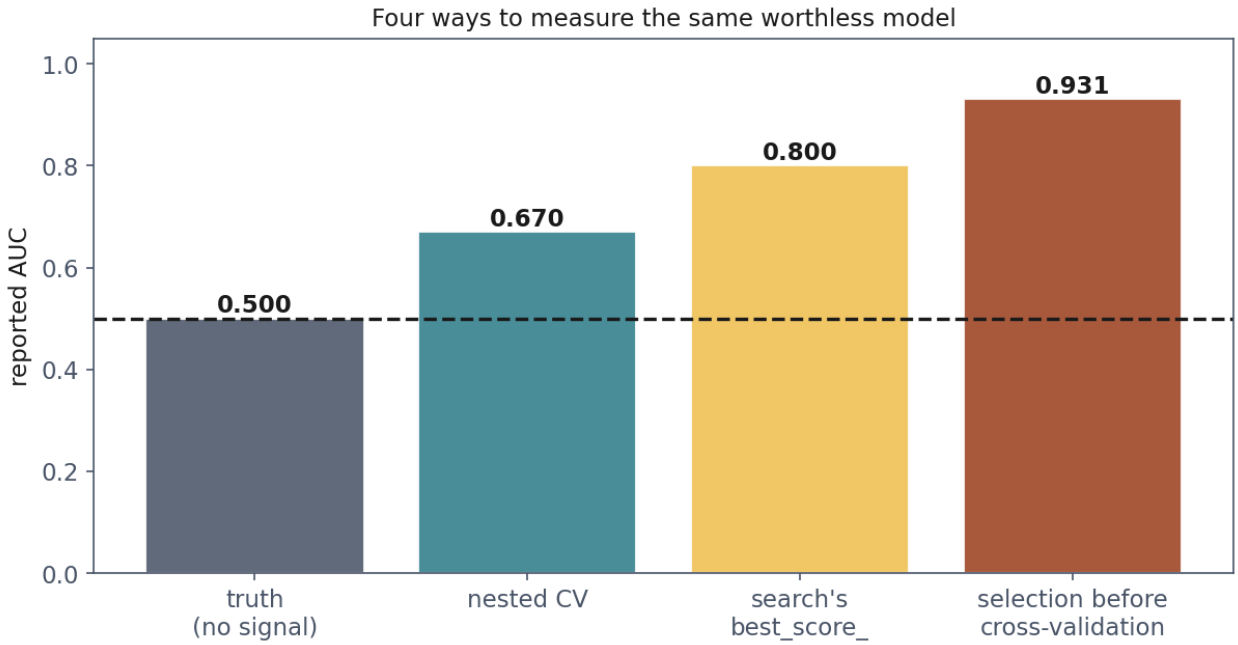
Outer round 1 of 5, drawn as notebook 3 runs it: look at the arrow. The 640 training drives are re-split into five new inner folds of 128, and the whole search runs there; the 160 outer test drives were never involved in choosing anything, which is why their score, 0.698, is one you may report. The bottom line averages the five rounds.

The inner loop may overfit its own data as much as it likes; the outer fold was never part of it. Nested cross-validation is what you use when you need an honest estimate of *"how well does my whole procedure, tuning included, do?"*

A predictable mistake: "0.670 is the score of the model I ship". The search ends with a refit and the procedure returns one number, so it reads as a model and its score. But the model refitted on all 800 drives is none of the five outer-round models that produced 0.670: the figure measures the procedure, search included, as section 4.2's 0.9514 measured a recipe.

It is not a way to choose hyperparameters — it produces k possibly different winners. It is a way to estimate what choosing costs. Choose on all the data afterwards, and report the nested figure.

What choosing cost. Against `best_score_`, the nested figure gives the optimism of the search: $0.7999 - 0.6699 = 0.1300$, larger than most of the differences anyone reports between methods. The four numbers of this section side by side:



Four numbers describing the same data, which contains no signal at all. Each step to the left is a piece of discipline, and each costs a chunk of the score you would otherwise have reported.

Why each step lands closer to 0.5. One rule reads the whole ladder: *a score is honest to the extent that the rows that measure it took part in no decision.* Each step to the left removes one decision taken while looking at the scoring rows:

Step	What the scoring rows had a hand in	As an exam
0.931, selection before cross-validation	choosing the **columns**: the selector saw their labels	the student saw the questions *and* the answers beforehand
0.800, `best_score_`	choosing the **configuration**: the same folds choose and score, and the luckiest on them wins	the student sat 25 mock exams and reports the best mark
0.670, nested cross-validation	**nothing**: every choice is made on the 640, every score on the 160 that took no part	the student prepares on mock exams, then sits an exam never seen
0.500, the truth		

Why 0.670 and not 0.500. The $0.670 - 0.500 = 0.170$ that remains is section 7.2's accident of the sample. The noise columns that happen to match the label match it in all 800 drives, the 160 outer test drives included: nested cross-validation separated choosing from scoring, but inside the same sample. The exam never seen still comes from the same question bank, with its quirks. Only an exam from another school — new data: a test set set aside before any search, or fresh rows — brings the number back to 0.5, as section 7.2's checks and atom 14 show.

> Each piece of discipline removes a decision taken on the scoring data, and each one costs score. Only new data removes the sample's accident too.

Checks, for whoever wants the detail. The nested figure is honest but not precise. Its five outer folds score 0.698, 0.852, 0.754, 0.487 and 0.558, a mean of 0.6699 with a standard deviation of 0.13: one fold lands below chance, another at 0.85. Each outer test fifth holds 160 drives and only about six failures, so each fold's AUC rests on a handful of positives — section 2.4's point about the rarer class. The search's own figures, as notebook 3 prints them to four decimals: best 0.7999, average 0.7265, worst 0.6840; the slides round them to three decimals.

---

8. Doing lesson 4's threshold honestly

The debt from section 1.1, now paid. Split three ways: **train** to fit, **validation** to choose, **test** to report once.

The data are Section 2.2's 800 drives, not lesson 4's 8,000, so the euros below are not comparable with lesson 4's 87,620 and 45,540: the bill is smaller because the test set is. The split: 200 drives (25%, 7 failures) set aside as the test set; of the remaining 600, 150 (5 failures) become the validation set and 450 the training set — 56%, 19% and 25% of the whole. The costs are lesson 4's, 140 EUR for a false alarm and 2,600 EUR for a missed failure.

One logistic regression, standardised first, is fitted once on the 450 training drives and serves every row below; the rows differ only in the threshold, and every bill is counted on the same 200 test drives. "Chosen" means a sweep: 200 thresholds from 0.01 to 0.9, each priced on a set's predicted probabilities, the cheapest kept.

Threshold	Missed × 2,600 + false alarms × 140	Cost on the 200 test drives
0.50, the default: nothing chosen	4 × 2,600 + 1 × 140	10,540 EUR
0.050, the sweep on the 150 **validation** drives	1 × 2,600 + 15 × 140	**4,700 EUR**
0.171, the same sweep on the 200 **test** drives	1 × 2,600 + 5 × 140	3,300 EUR

Both chosen thresholds catch 6 of the 7 failures; they differ only in false alarms, 15 against 5. The 4,700−3,300 = 1,400 EUR between them — 42% more than the cheating figure, 1,400÷3,300 = 0.42 — is ten healthy drives that this particular test set happened to let 0.171 leave alone.

Look at 0.050. It sits next to lesson 4's $t^* = 140/2{,}740 = 0.051$, the threshold the two costs give before any data are seen, provided the probabilities are calibrated. With five failures in the validation set part of the agreement is luck, but the lesson stands: the formula was already the honest answer. What needed paying was the 0.078 lesson 4 found by sweeping its test set.

The honest threshold is worse on the test set than the cheating one. **It is supposed to be.** The cheating figure describes a threshold tuned to this particular test set and there is no reason it survives contact with the next two hundred drives; the honest figure is an unbiased estimate of what happens next.

A predictable mistake: "the debt is paid, so the number improves." The instinct is reasonable — doing things properly ought to be rewarded — but paying the debt makes the reported

number *worse*, because it removes the luck the cheating number had borrowed from the test set.

Note the cost of doing it properly: we now train on 56% of the data instead of 75%, because the validation set has to come from somewhere. On a small dataset that hurts — which is the argument for choosing the threshold by cross-validation on the training portion instead, and keeping the test set whole.

Three sets, three jobs — the principle the whole lesson rests on. Training fits the parameters. Validation — a held-out set, as here, or the cross-validation folds of section 4 — makes every choice: model, hyperparameters, features, threshold. The test set reports, once. Change anything after looking at the test set and it has become a validation set: say so in the report, or get fresh data. Everyone looks twice sometimes; a stated weakness costs a little credibility, an unstated one all of it.

---

9. Reproducibility

9.1 Seeds

Fix every seed, and write them down:

```
train_test_split(..., random_state=0)
StratifiedKFold(..., shuffle=True, random_state=0)
LogisticRegression(..., random_state=0)     # solvers that sample
np.random.default_rng(0)                    # your own generation
```

But a fixed seed buys repeatability, not stability. Thirty seeds of the train/test split (600 drives to fit, 200 to test), same model, gave scores from 0.913 to a perfect 1.000 — and every one of those runs is exactly reproducible by anyone with the same code.

`AUC 1.000 (random_state=3)` is fully reproducible and thoroughly misleading. `AUC 0.951 ± 0.019 over 5-fold cross-validation, seed 0` is reproducible and honest.

9.2 The rest of it

- **Record library versions.** `numpy`, `pandas` and `scikit-learn` change defaults between releases; a result nobody can reproduce in two years was not reproducible.
- **Name what you could not fix** — parallel computations that finish in a different order, graphics processing unit (GPU) non-determinism.
- **Commit the code that produced the number**, not a cleaned-up version of it.

The course container exists for this reason: it pins the environment so that your result and your marker's result are the same result.

---

10. Summary

- **A test score is a measurement with an error bar.** Holding data out makes it unbiased, not precise. On 800 drives the same model scored **0.885 to a perfect 1.000** depending only on the seed.

- **When one class is rare, its count governs the precision** of recall and, in practice, of AUC. Ask how many positives are in the test set.
- **Selecting on one split selects noise.** Three equally good models won 89, 42 and 38 of 200 splits outright, 31 ties, by a median margin of 0.0015.
- **Cross-validation** rotates a validation fold, estimates a *procedure*, is usually slightly pessimistic, and s ÷ √k understates its uncertainty by about half — so quote the spread, not a confidence interval.
- **bias² + variance + noise is an identity**, verified to 3×10^{-12}. The noise floor never moves, and a test error well below it means leakage or a mistake in the evaluation.
- **The best model depends on how much data you have.**
- **Without the truth, two errors still diagnose.** A training error too high for the task points to bias; a validation error far above the training error points to variance. The training error hides variance; it does not measure it.
- **Learning curves prescribe the fix.** Training and validation curves that meet at a poor score mean bias, and more data will not help; a gap between them means variance: more data helps only while the validation curve is still rising by more than its spread; once it is flat, a stronger penalty or fewer features.
- **Cross-validation is honest only about what runs inside it**: select and tune inside it, and report the nested figure. Three things it cannot remove: the sample's chance correlations (selection from 2,000 noise columns still scored 0.658 done fold-honestly, section 7.2; a test set set aside first exposes them, fewer candidates or more rows shrink them); the luck of choosing (`best_score_` 0.7999 against a nested 0.6699, sections 7.3-7.4); and rows that are not independent, unless each drive's readings stay in one fold (`GroupKFold`, section 4.6).
- **`best_score_` is the maximum of many noisy estimates.** Nested cross-validation measures the search.
- **Split along the axis you must generalise across** — `GroupKFold` when rows share an entity.
- **Choose on validation data, report on test data, touch the test set once.**

What belongs in a report

The metric, **the spread**, and how it was estimated. How many **positives** the test set held. Every **seed** and every **library version**. And which choices were made on which data.

Those four items map onto the four failures in this lesson: a number without a spread hides the lottery; a metric without a positive count hides why the spread is wide; a missing seed makes it unreproducible; and an unstated selection procedure hides the optimism.

Read papers with the same list. A single figure with no spread, no fold count and no statement about how the hyperparameters were chosen leaves all four questions open — and section 7.3 suggests how large the correction tends to be.

Homework

`Exercises/05_experimental_methodology.md`, set on Friday 13 November 2026 and discussed on **Friday 20 November 2026**.

Notation used in this lesson

Symbol	Meaning
S, S_j	the dataset; its j-th fold
$A(S)$	the model our procedure produces from dataset S
f, $\hat{f}$, $\bar{f}$	the truth; a fitted model; the average fitted model
R, $\hat{R}$	expected risk; its empirical estimate
$\hat{R}_{CV}$	the cross-validated estimate
$\mathcal{D}$	the unknown distribution the data comes from
σ^2	the irreducible noise variance
ρ	the correlation between round scores
m, k	number of examples; number of folds